IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are

PART I

Collection and Preparation
Lec 2-5

PART II

SQL
Lec 6-8

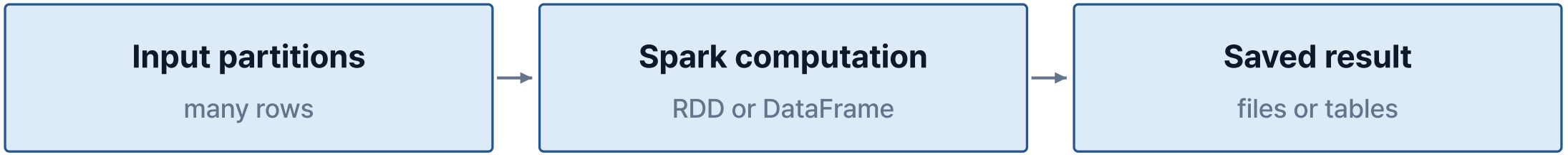
NoSQL
Lec 9-10

Big Data
Lec 11-13

PART III

Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing



A distributed program still needs infrastructure, storage and a way to run again

Why Managed Infrastructure

usenix ASSOCIATION Conferences Sign In

SRECON AMERICAS ATTEND PROGRAM PARTICIPATE SPONSORS ABOUT

Why Are Distributed Systems So Hard? ⟶ You are NOT alone!

Wednesday, March 27, 2019 - 5:00 pm–5:30 pm
Denise Yu, Pivotal

Abstract:
Distributed systems are known for being notoriously difficult to wrangle. But why? This talk will cover a brief history of distributed databases, clear up some common myths about the CAP theorem, dig into why network partitions are inevitable, and close out by highlighting how a few popular consensus algorithms mitigate the risks of operating in a distributed fashion and the importance of considering human factors to fully understand the systems we build. Almost all slides will contain original illustration featuring mischievous cats masquerading as sysadmins. By the end of this talk you will have a better understanding of the design trade-offs involved in architecting for distributed systems, and hopefully, be inspired to start doodling tech concepts!

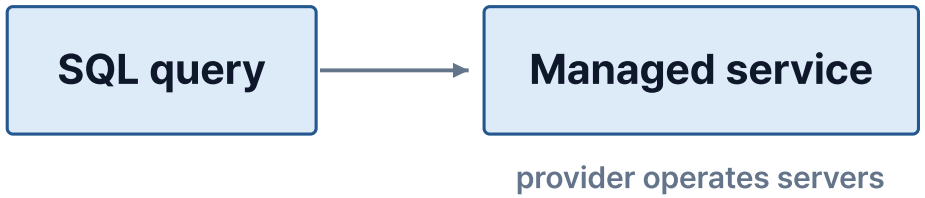


Denise is a software engineer who occasionally wears a product management hat at Pivotal R&D in Toronto. Denise has previously delivered conference talks on topics ranging from continuous delivery to functional programming to scaling company culture. She enjoys learning about distributed systems, release engineering, and low-level Linux kernel programming, and when she's not coding, she is often doodling sketch notes that break down technical concepts into digestible pieces at deniseyu.io/art.

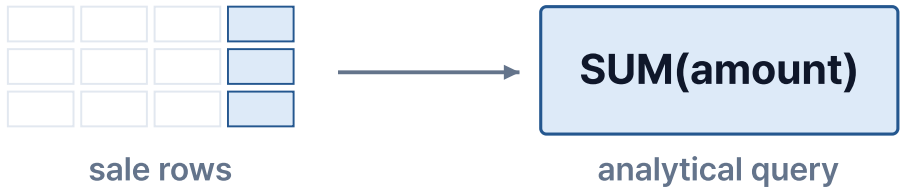
Writing the computation leaves the problem of operating the system

Outline: Big Data II

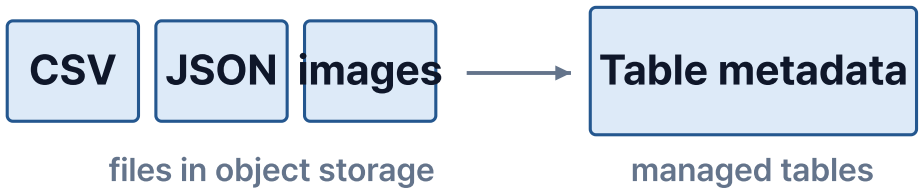
01 Cloud and Serverless



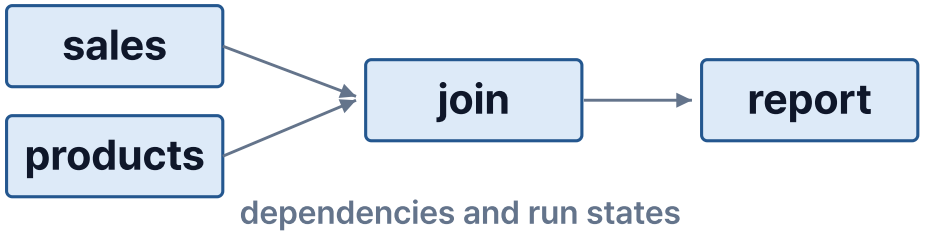
02 Analytical Warehouses



03 Lakes and Lakehouses

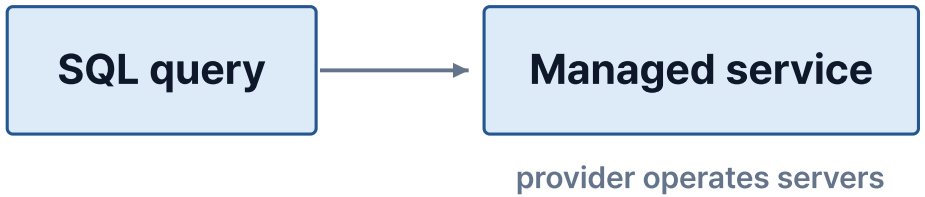


04 Data Orchestration

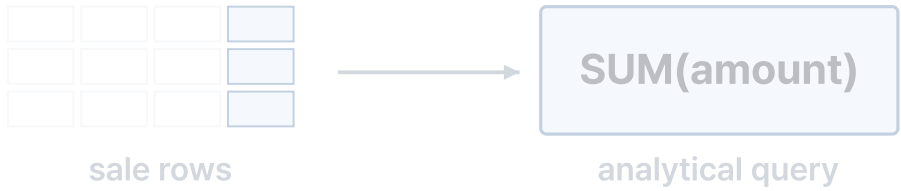


01 · Cloud and Serverless

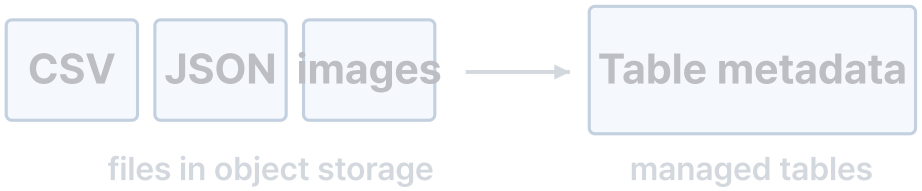
01 Cloud and Serverless



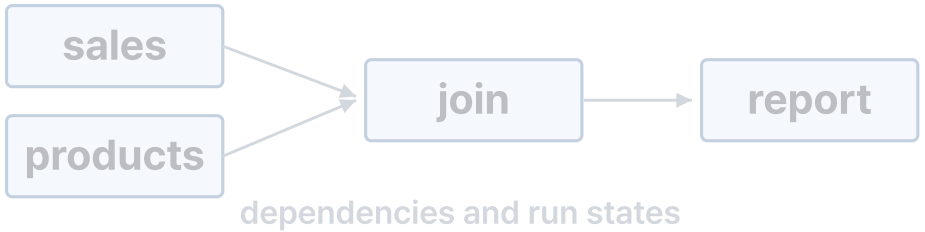
02 Analytical Warehouses



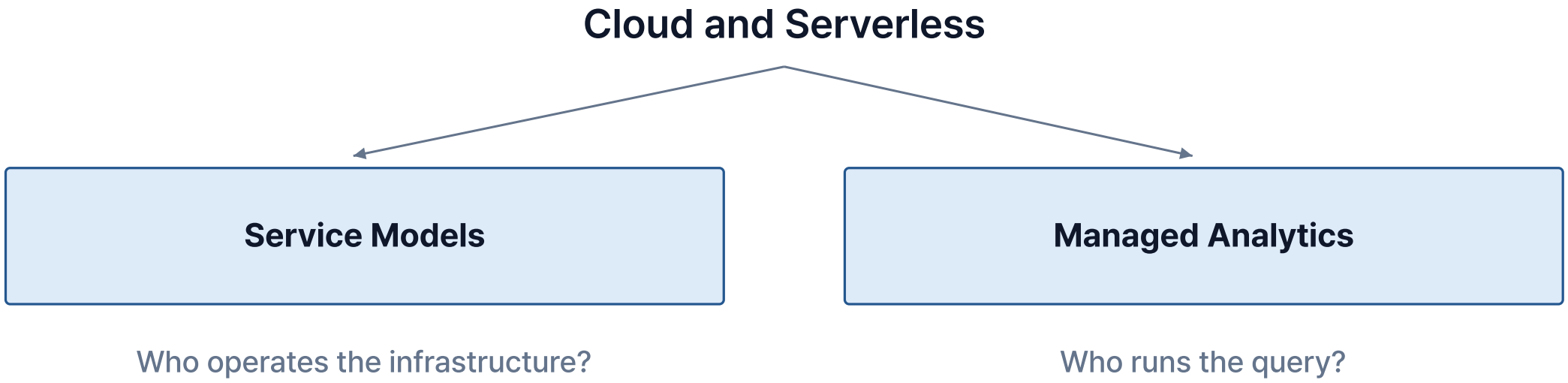
03 Lakes and Lakehouses



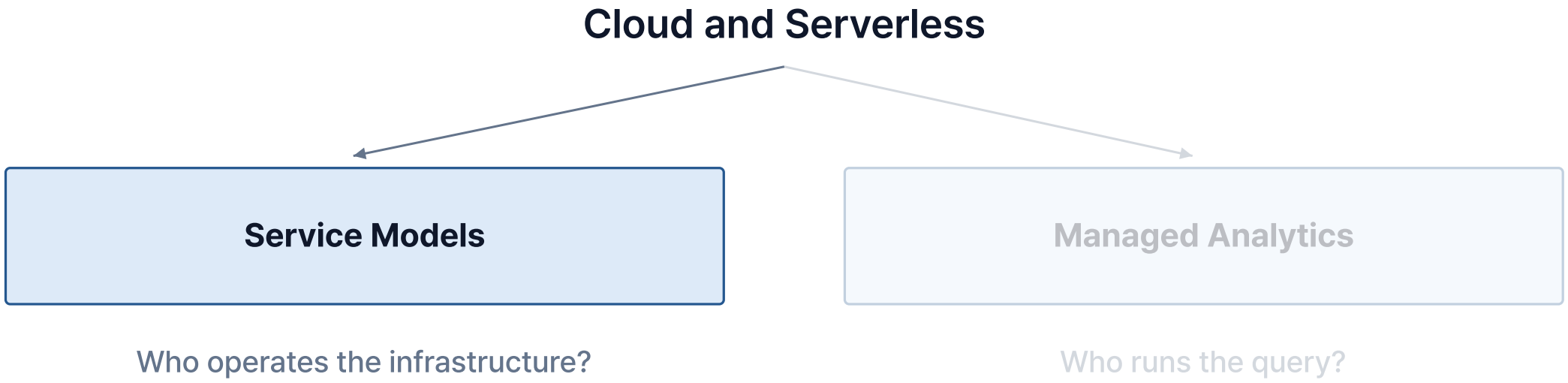
04 Data Orchestration



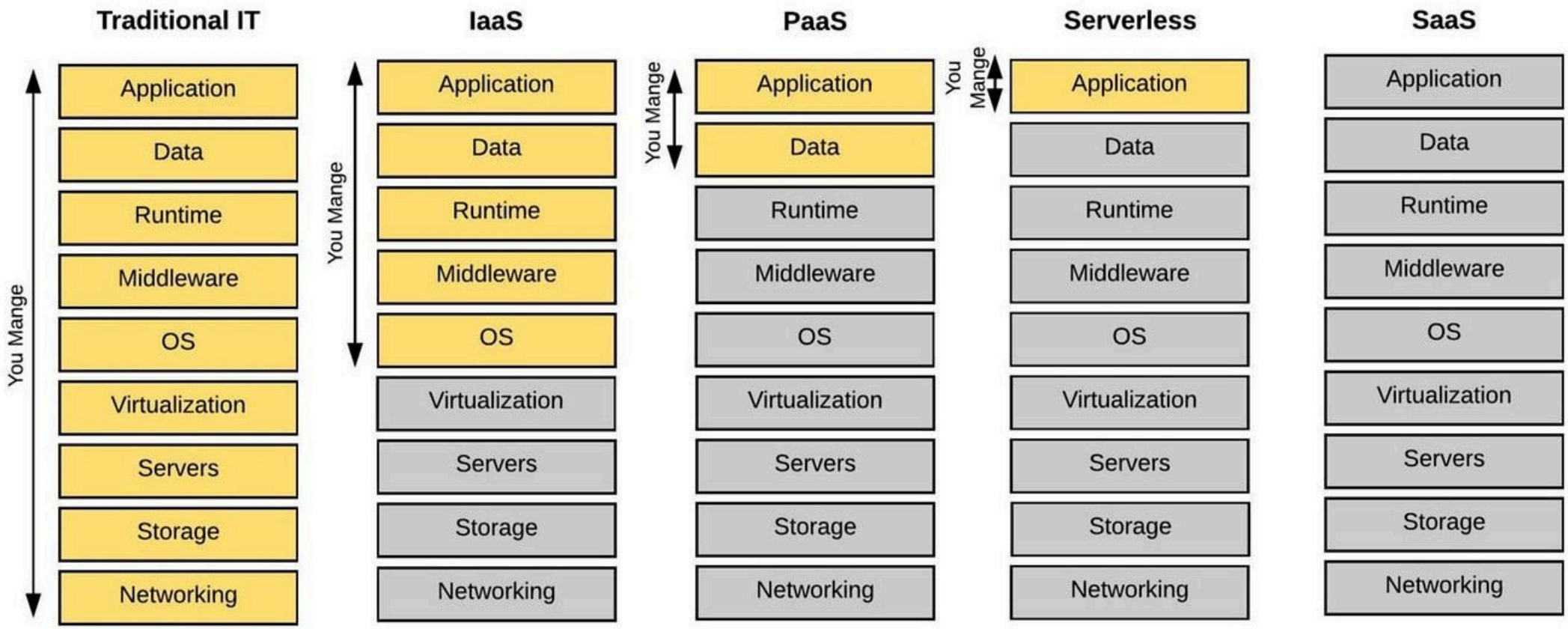
Cloud and Serverless



§ Service Models



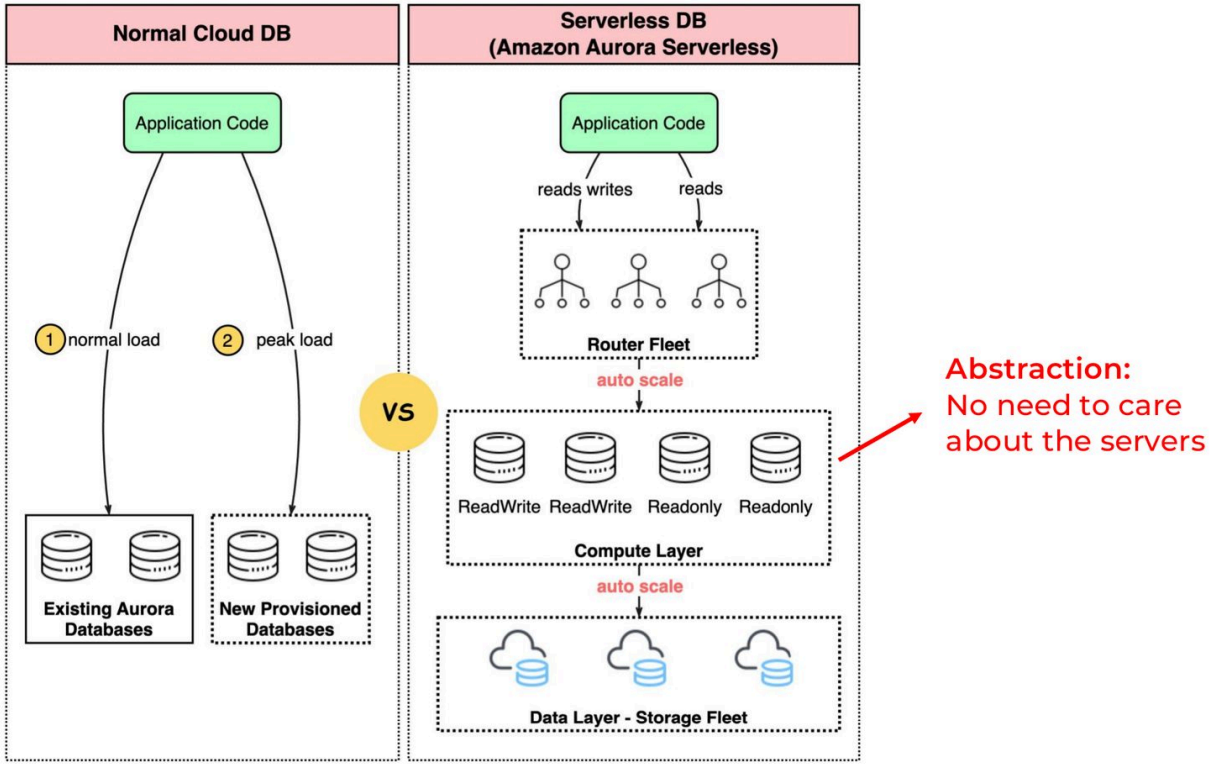
Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service



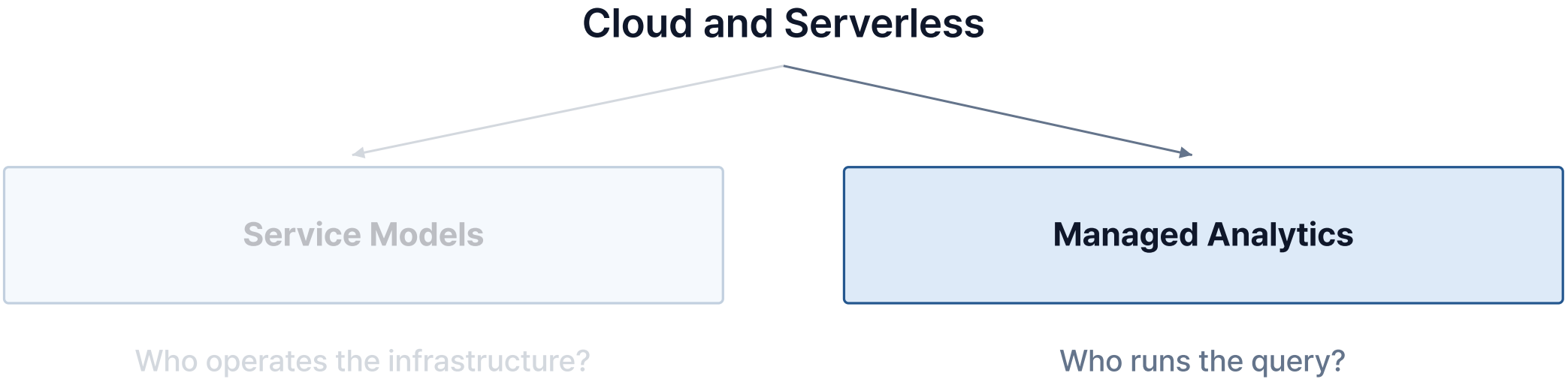
The service hides server allocation behind an application interface

The Remaining Responsibilities

Provider manages	User still decides
Servers and service maintenance	Data meaning and schema
Service execution infrastructure	Queries and access permissions
Service-level scaling mechanisms	Workload design and cost controls

Managed infrastructure still requires deliberate data, access and workload choices

§ Managed Analytics



Milestones in Managed Analytics

Milestone	What became easier
2000s: Hadoop MapReduce	Distribute batch computations
2009: Spark begins at Berkeley	Express reusable computation chains
2010 preview: BigQuery	Submit analytics to a managed service
Cloud adoption	Use managed compute and shared storage

Managed services add an operational choice; Spark remains useful

BigQuery as a Managed Service



Ingest

- Batch & Streaming
- Structured & Semi-structured



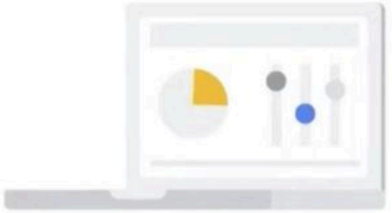
Store

- Up to Petabytes of data
- Datasets & tables
- Federate data



Analyze

- Super-fast SQL queries
- Aggregate
- Transform with DML



Visualize

- Export and connect to other tools more easily

BigQuery manages the infrastructure around loading tables and running analytical queries

Spark and BigQuery

Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

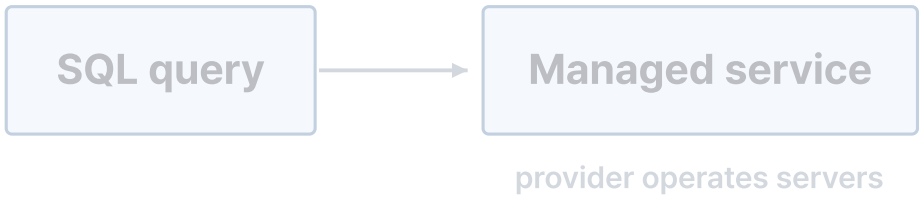
Managed Analytics Services

Service	Role
BigQuery (Google Cloud)	Managed analytical tables and SQL execution
Snowflake	Managed data platform on public clouds
Amazon Redshift	Managed analytical warehouse
Amazon Athena	Serverless SQL queries over S3 data

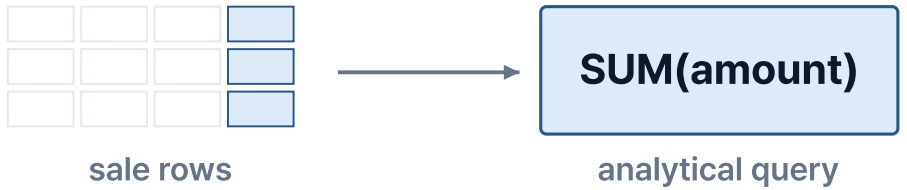
Similar SQL interfaces can sit over different data and service architectures

02 · Analytical Warehouses

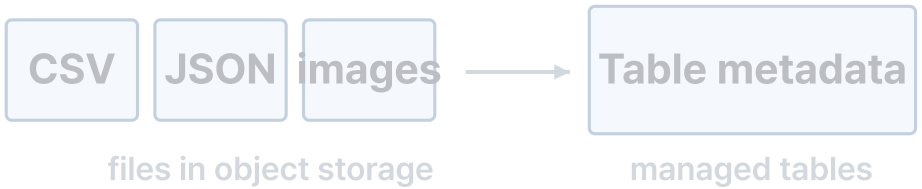
01 Cloud and Serverless



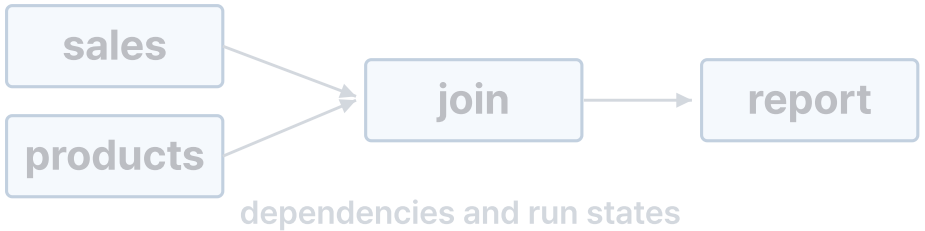
02 Analytical Warehouses



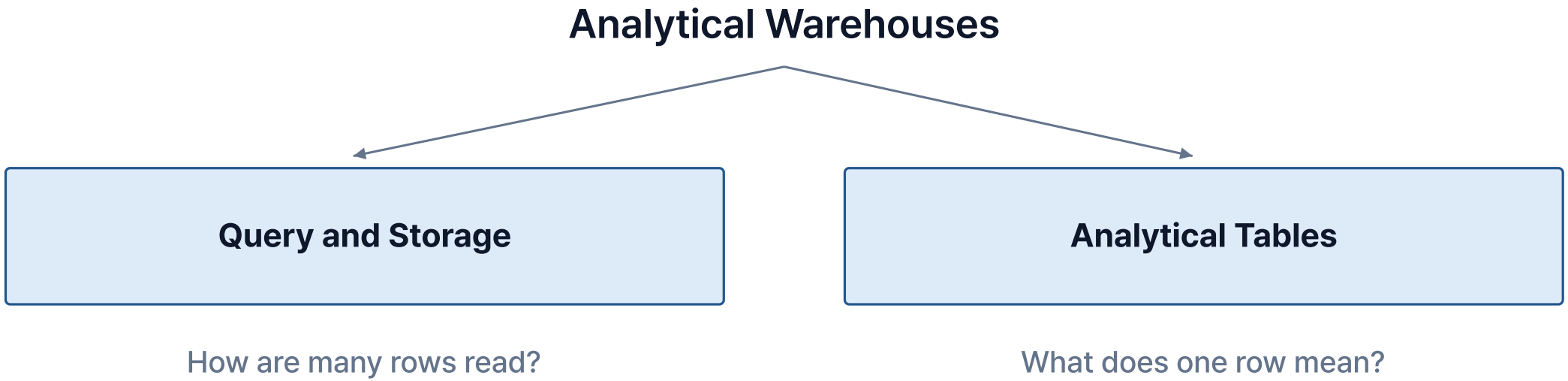
03 Lakes and Lakehouses



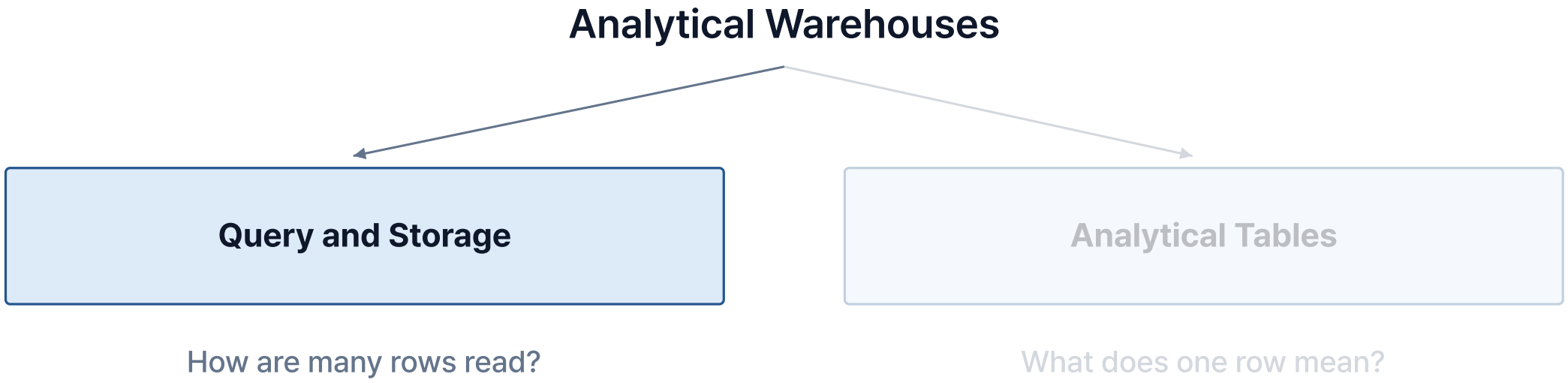
04 Data Orchestration



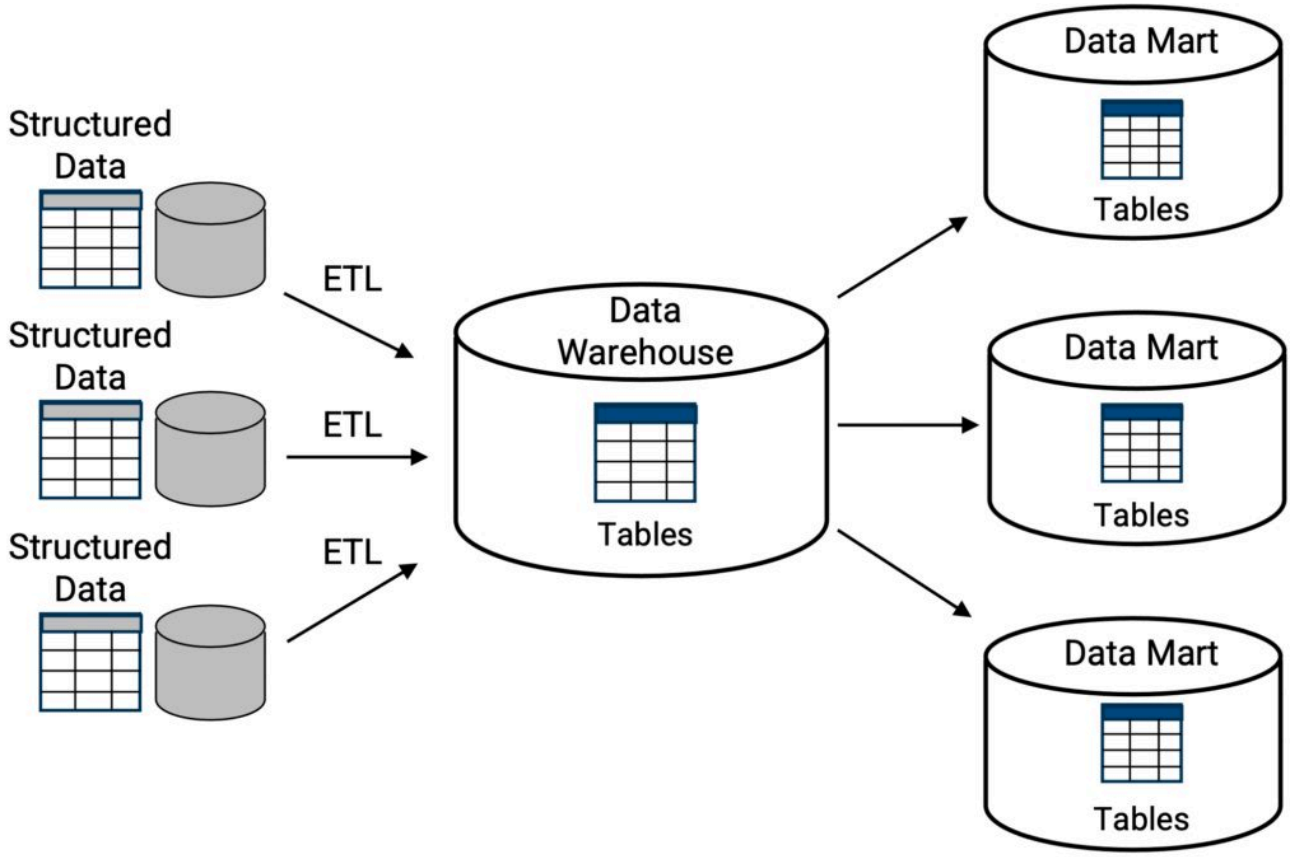
Analytical Warehouses



§ Query and Storage



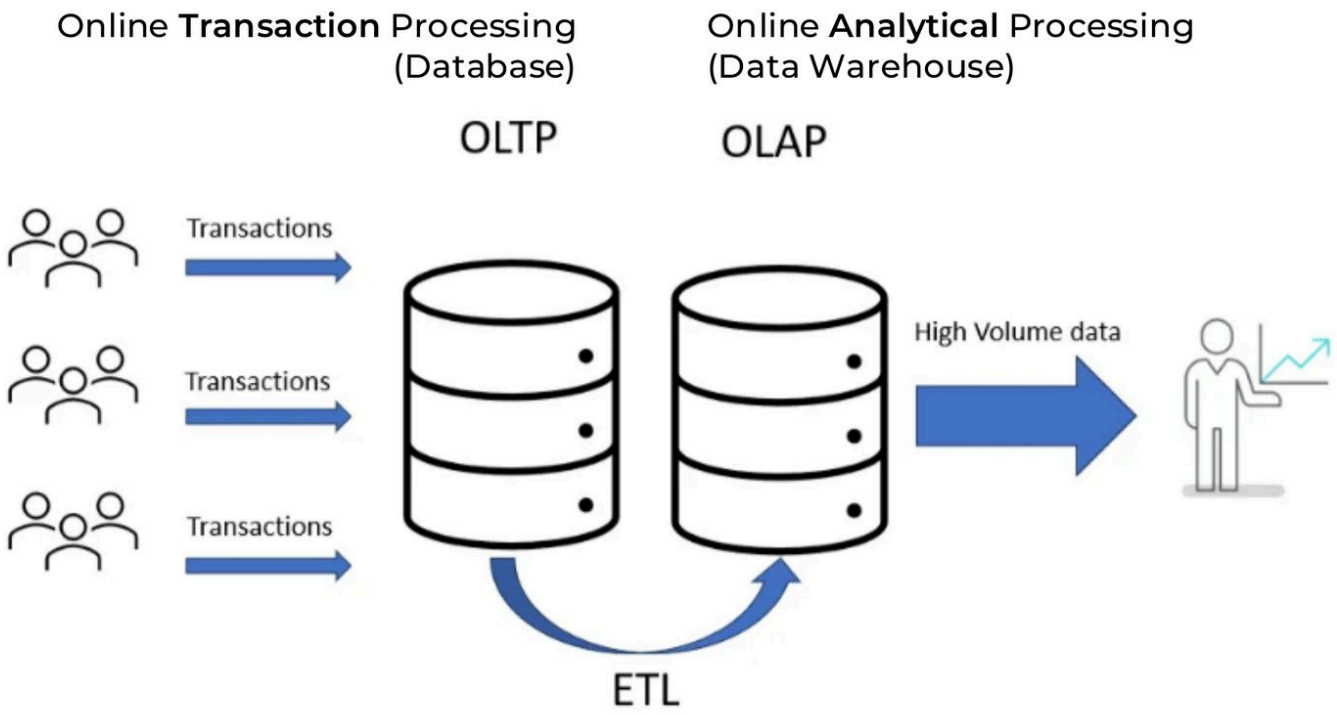
Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

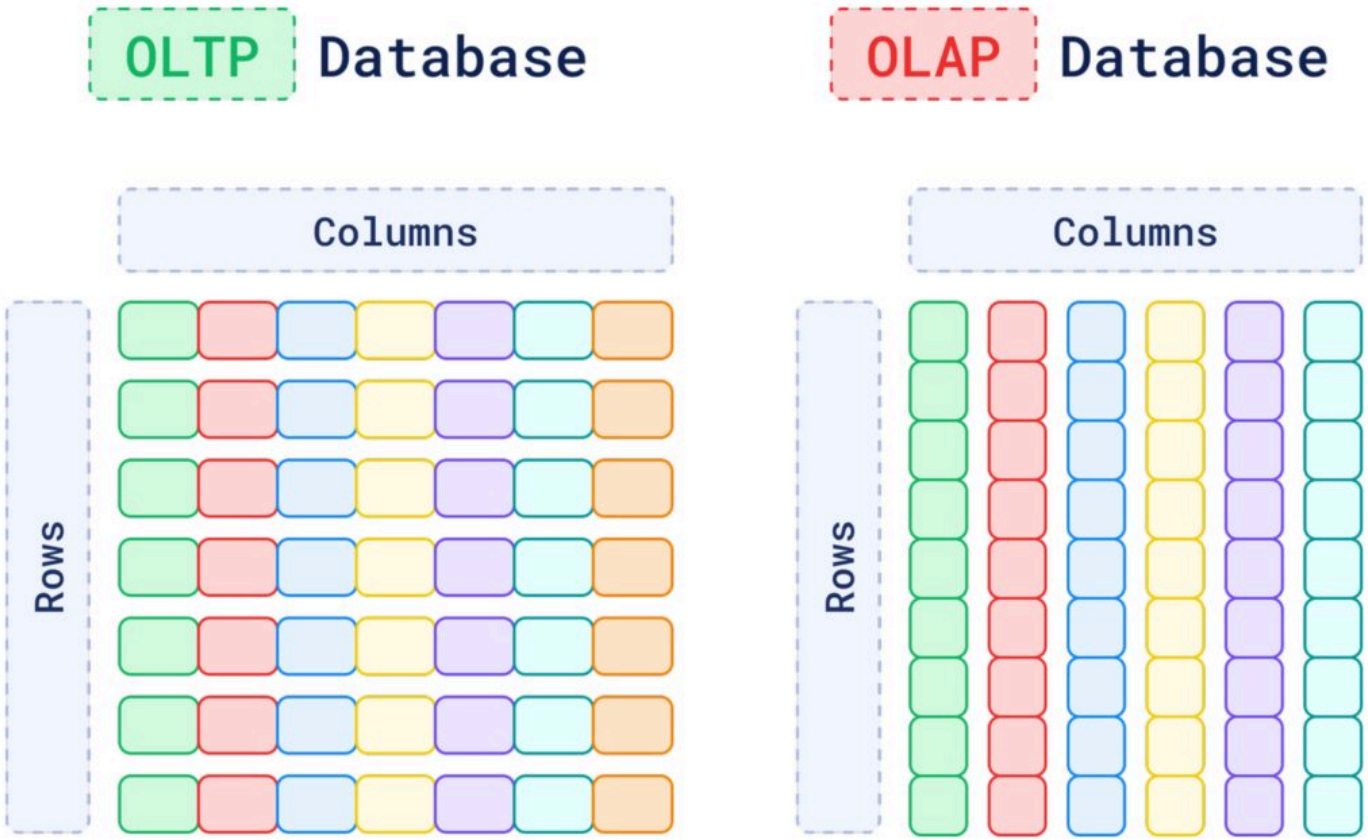
OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



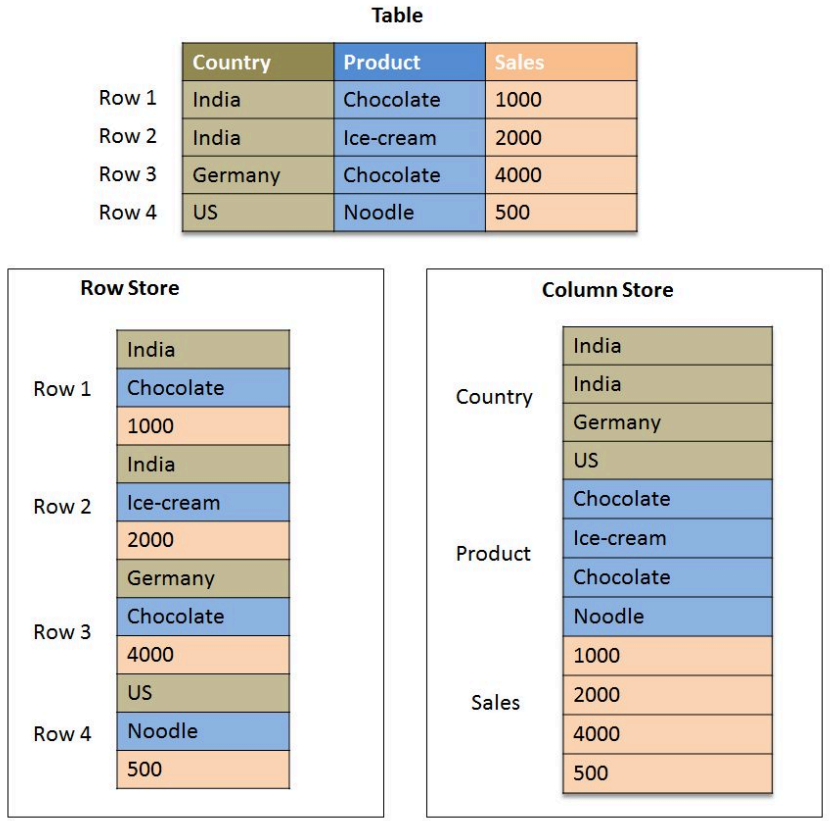
Small transactional updates and large analytical scans place different demands on storage

Row and Column Layouts



The same logical table can group values by row or by column

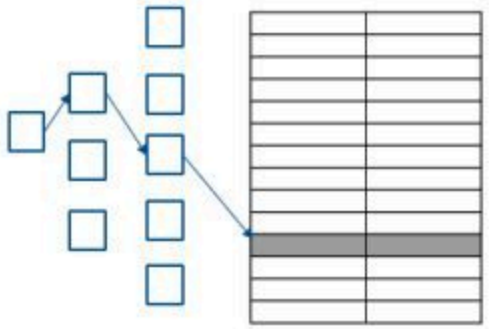
The Same Values on Disk



Twelve values retain their meaning while their physical order changes

Point Reads and Analytical Scans

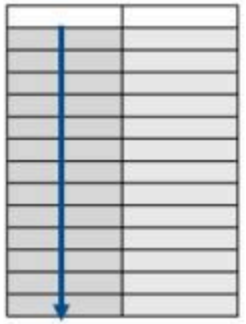
Row-oriented storage & execution



```
SELECT * FROM orders
WHERE orderid = $1
```

At scale: Fast on OLTP, Slow on OLAP

Column-oriented storage & execution



```
SELECT productid, count(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC LIMIT 10
```

At scale: Slow on OLTP, Fast on OLAP

Choose a layout around the values that the workload needs to read

Sales Scan in Row Layout

sale_id	date	product	amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

full rows: 16 values read

This full-row scan reads sixteen values to compute a sum of four amounts

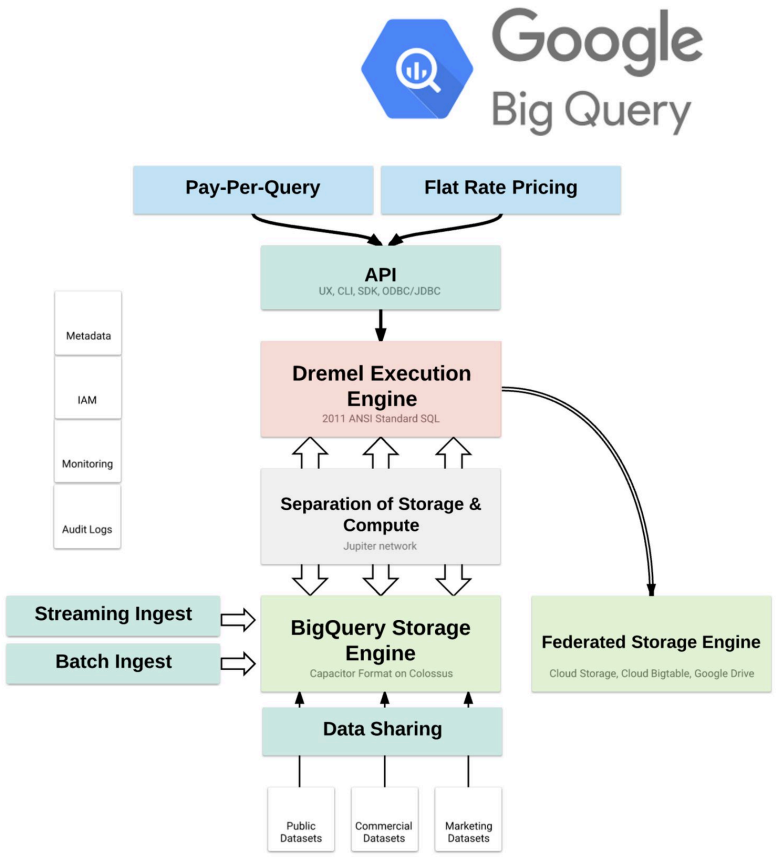
Sales Scan in Column Layout

sale_id	date	product	amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

amount column: 4 values read

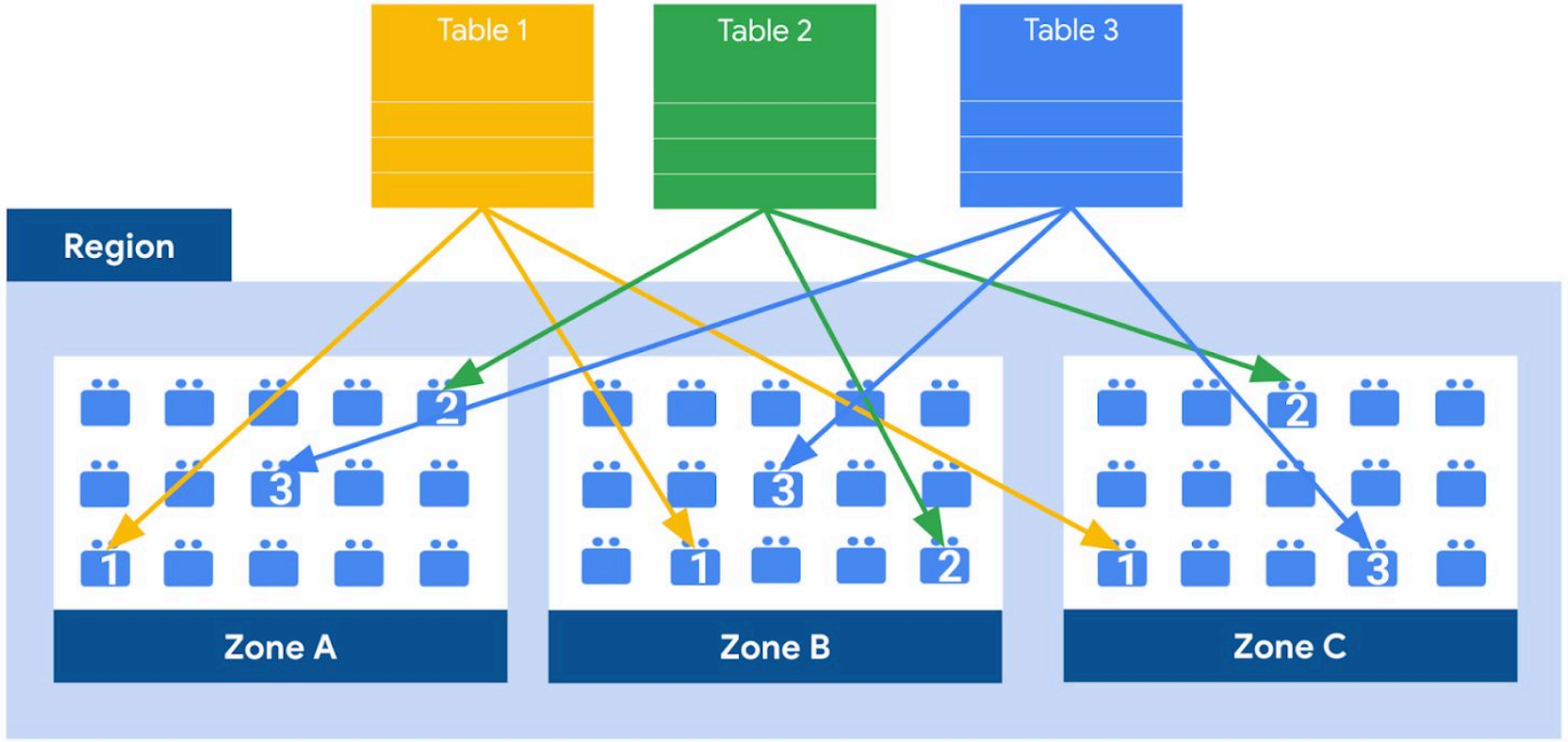
The column scan reads four amounts and returns the same total of 1000

BigQuery Storage and Compute



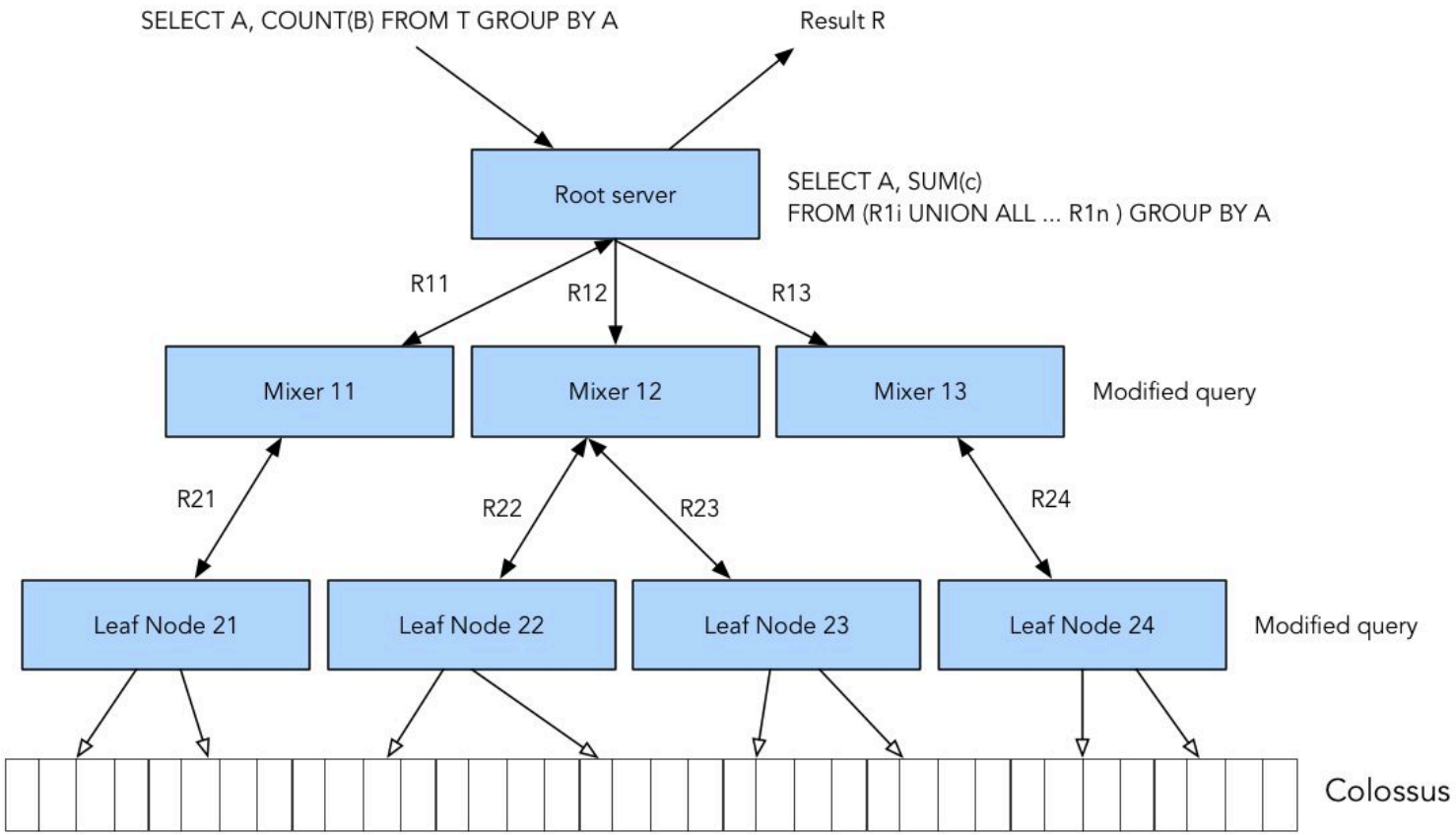
Persistent table storage and query execution are managed as separate layers

Managed Fault Tolerance



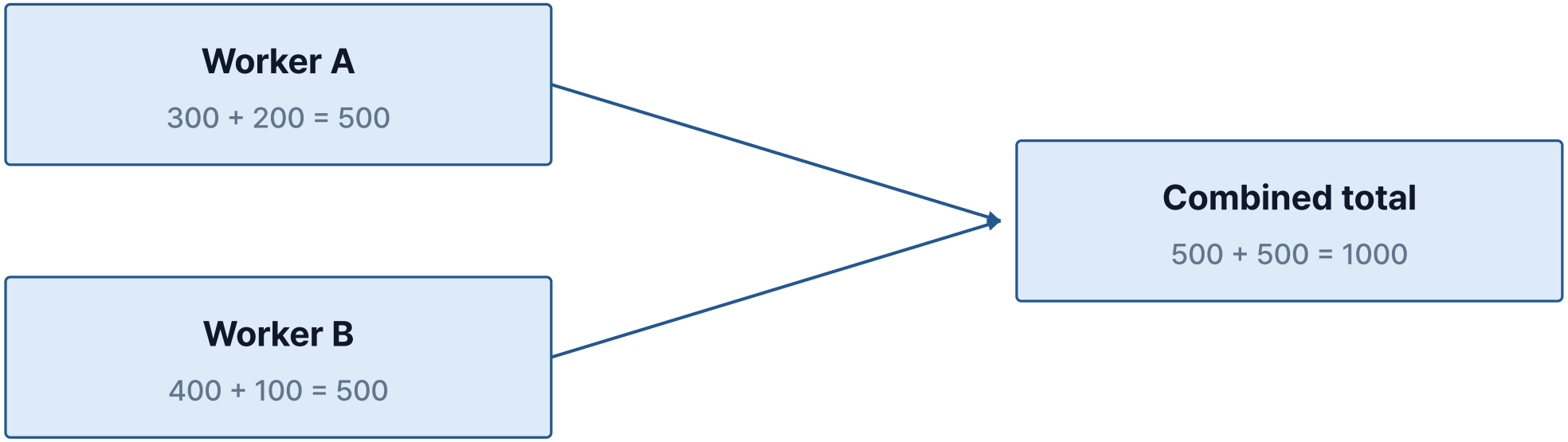
Replicated storage protects against infrastructure failures rather than incorrect analytical logic

Distributed Query Execution



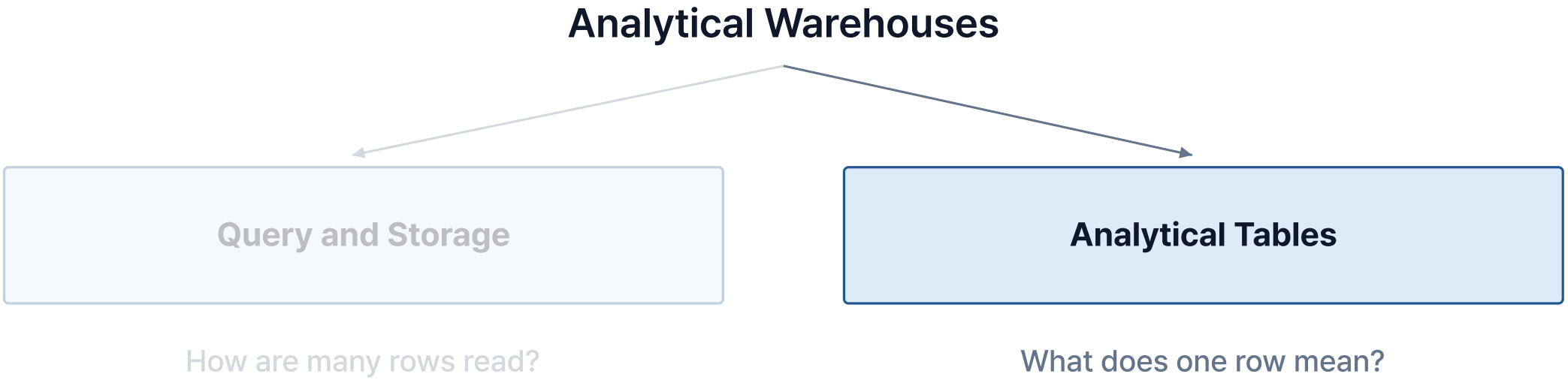
Workers read portions of the table and combine partial results

Partial Sums for Four Sales



Two partial sums produce the same total as one scan over all four sales

§ Analytical Tables



Recap: Four Sales

sale_id	sale_date	product_id	amount
1	2025-10-02	P1	300
2	2025-10-02	P2	200
3	2025-10-03	P1	400
4	2025-10-03	P2	100

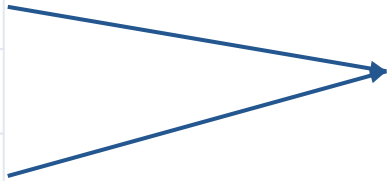
Four sale events can support more than one analytical grouping

Facts and Dimensions

A fact records an event or measurement; dimensions describe its context

fact_sales: one row per sale

sale_id	product_id	amount
1	P1	300
2	P2	200
3	P1	400
4	P2	100



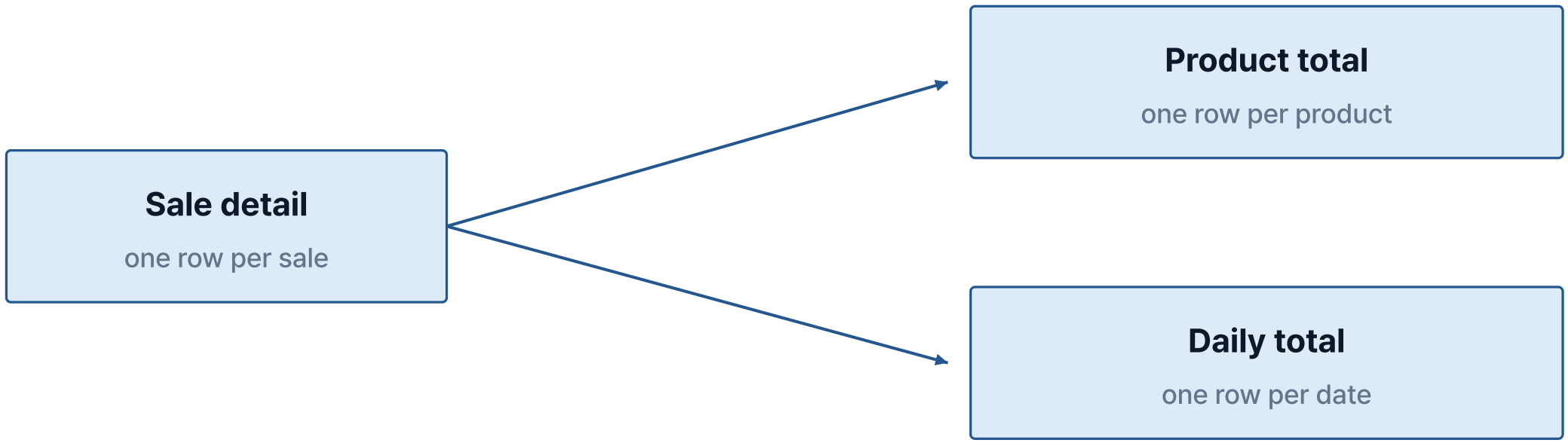
dim_product: one row per product

product_id	name
P1	Chocolate
P2	Ice-cream

product_id links a sale to its product name

Two sales of P1 share one product description

Grain of an Analytical Table



Changing the grouping changes what one result row represents

Product Sales in SQL

```sql
SELECT p.name,SUM(s.amount) AS total
FROM fact_sales AS s
JOIN dim_product AS p USING(product_id)
GROUP BY p.product_id,p.name ORDER BY p.product_id;
```

name	total
Chocolate	700
Ice-cream	300

The same four facts produce Chocolate 700 and Ice-cream 300

Demo: Analytical Questions

Local SQLite exercise: the analytical query, rather than BigQuery infrastructure

1. Return total sales for each product name
2. Keep only October 3 and compare the product totals

```
-- write your query here, then Run
```

QUERY RESULT

Write a query on the left, then Run

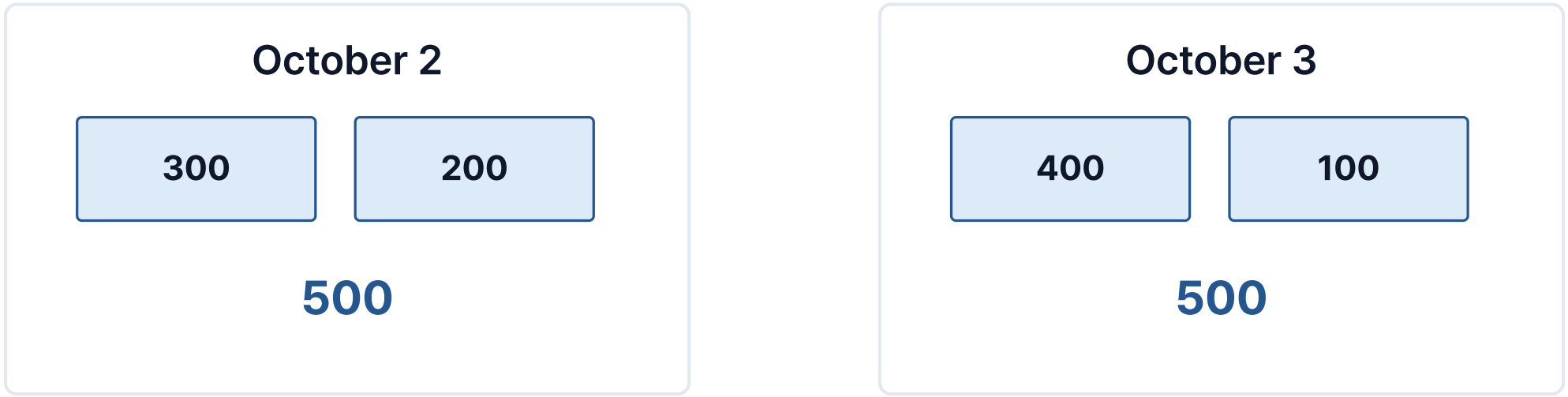
Answer: October 3 Product Sales

```sql
SELECT p.name,SUM(s.amount) AS total
FROM fact_sales AS s
JOIN dim_product AS p USING(product_id)
WHERE s.sale_date='2025-10-03'
GROUP BY p.product_id,p.name ORDER BY p.product_id;
```

name	total
Chocolate	400
Ice-cream	100

The date filter changes the contributing sales, while each row remains one product

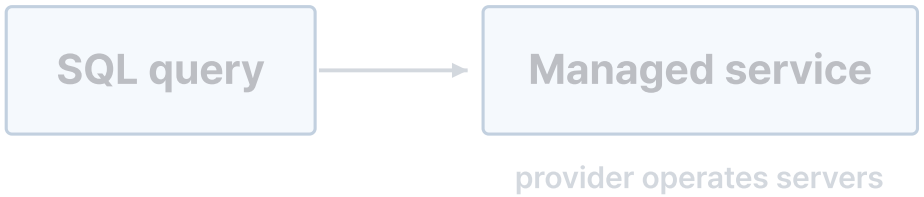
Date-Partitioned Sales



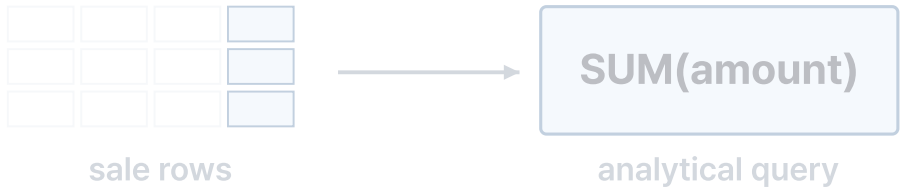
A date filter can skip unrelated partitions instead of scanning all sales

03 · Lakes and Lakehouses

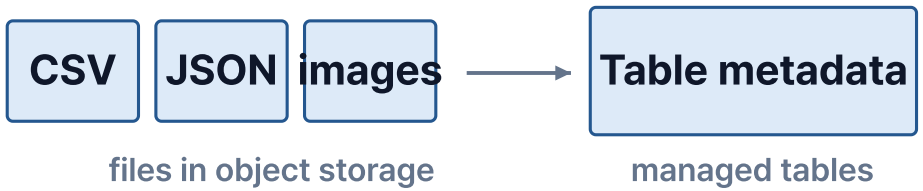
01 Cloud and Serverless



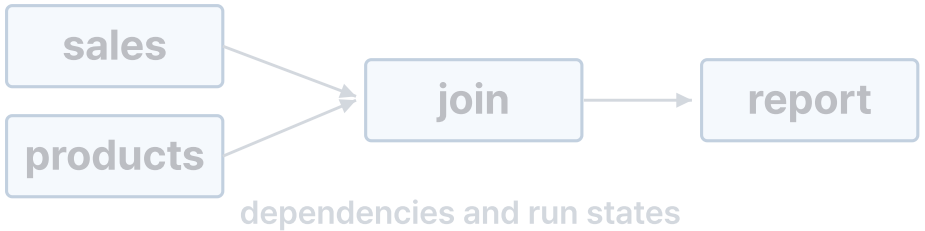
02 Analytical Warehouses



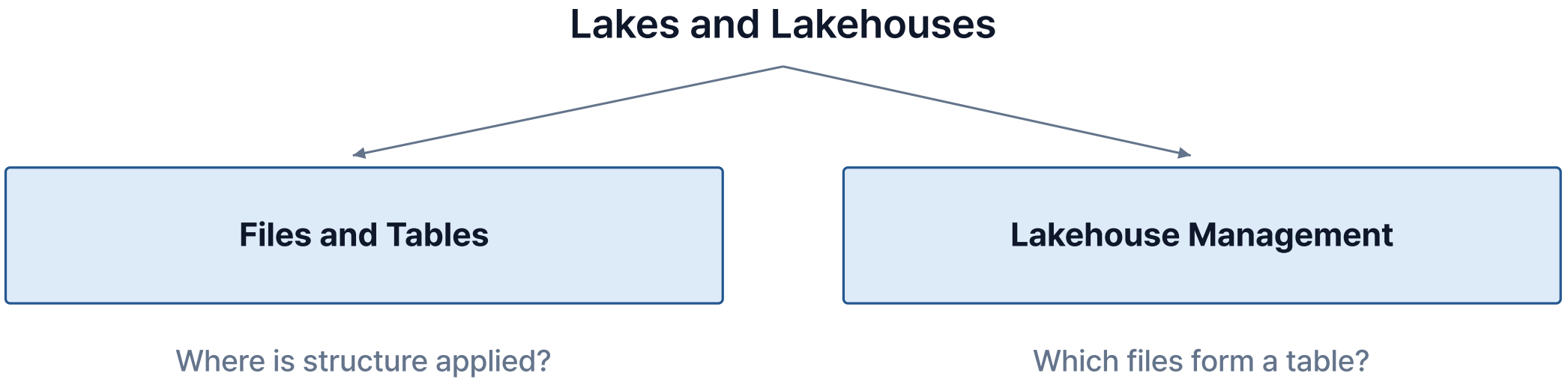
03 Lakes and Lakehouses



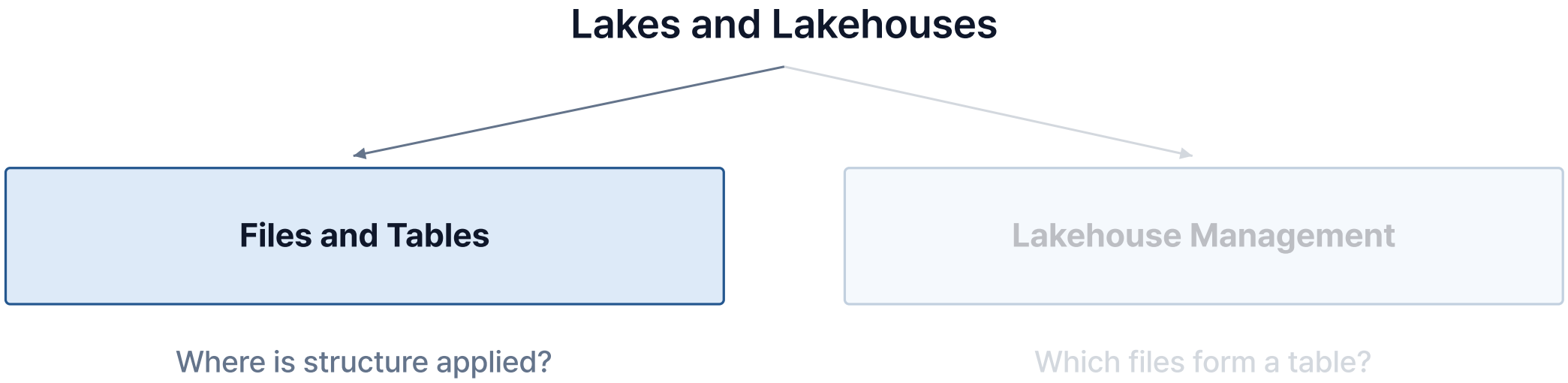
04 Data Orchestration



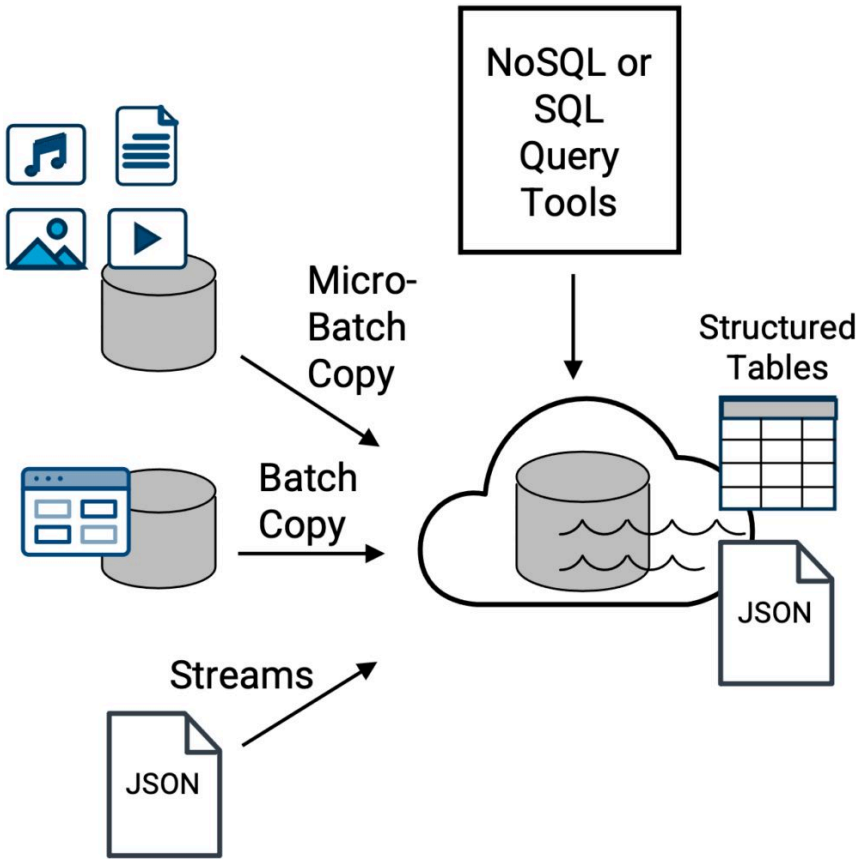
Lakes and Lakehouses



§ Files and Tables



Why Keep a Data Lake



Keep diverse source files for later analytical uses

Warehouse, Mart and Lake

Store	Organizing unit	Typical purpose
Warehouse	Integrated analytical tables	Consistent organizational analysis
Data mart	A subject or department subset	Focused reports
Lake	Files and datasets in varied formats	Preserve and reuse diverse inputs

A mart narrows the analytical scope; a lake broadens the retained source material

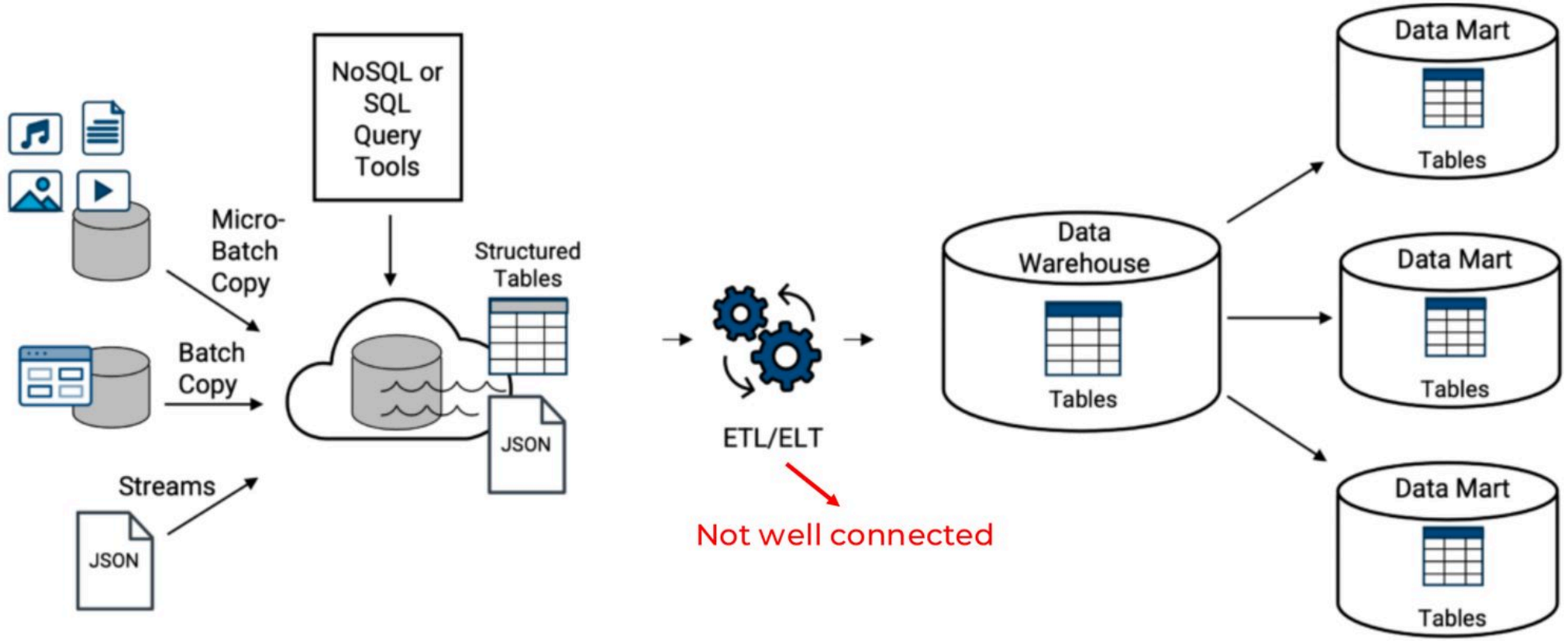
Schema on Read

Files can arrive first, but queries still need structure, types and meaning

```
{"sale_id":3,"product_id":"P1","amount":"400"}
{"sale_id":4,"product_id":"P2","amount":"100"}
```

The string "400" needs an explicit numeric interpretation before summing

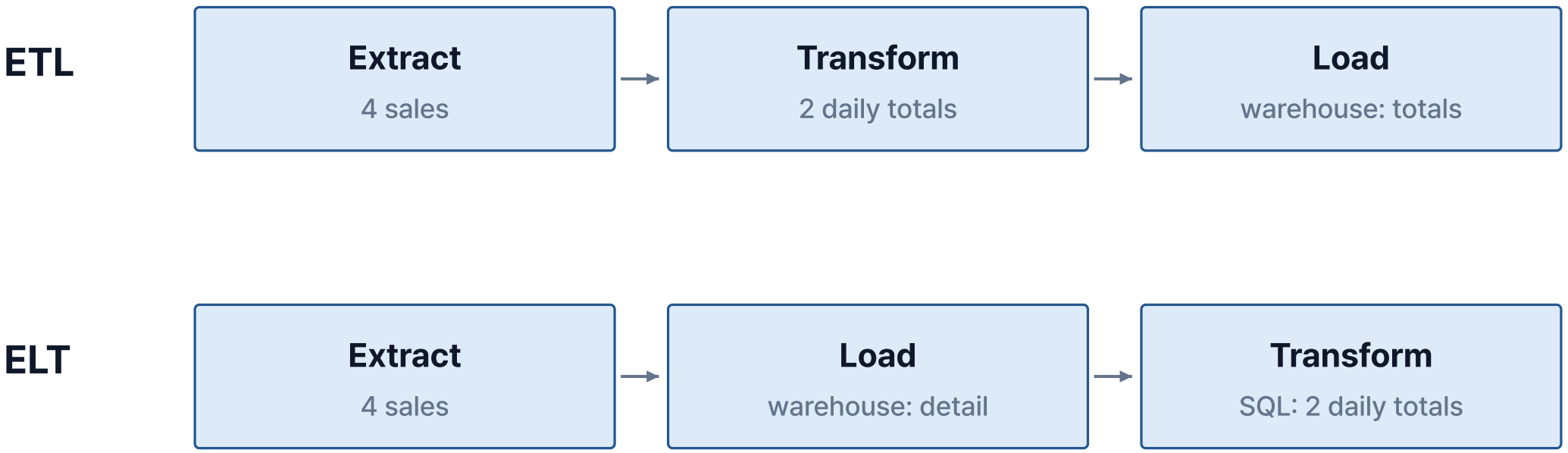
From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

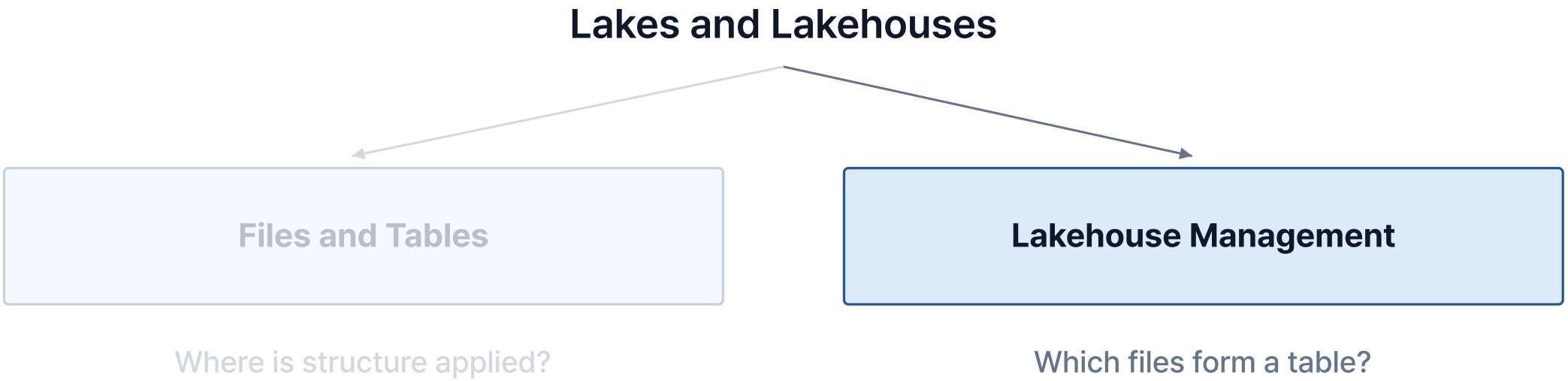
ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)

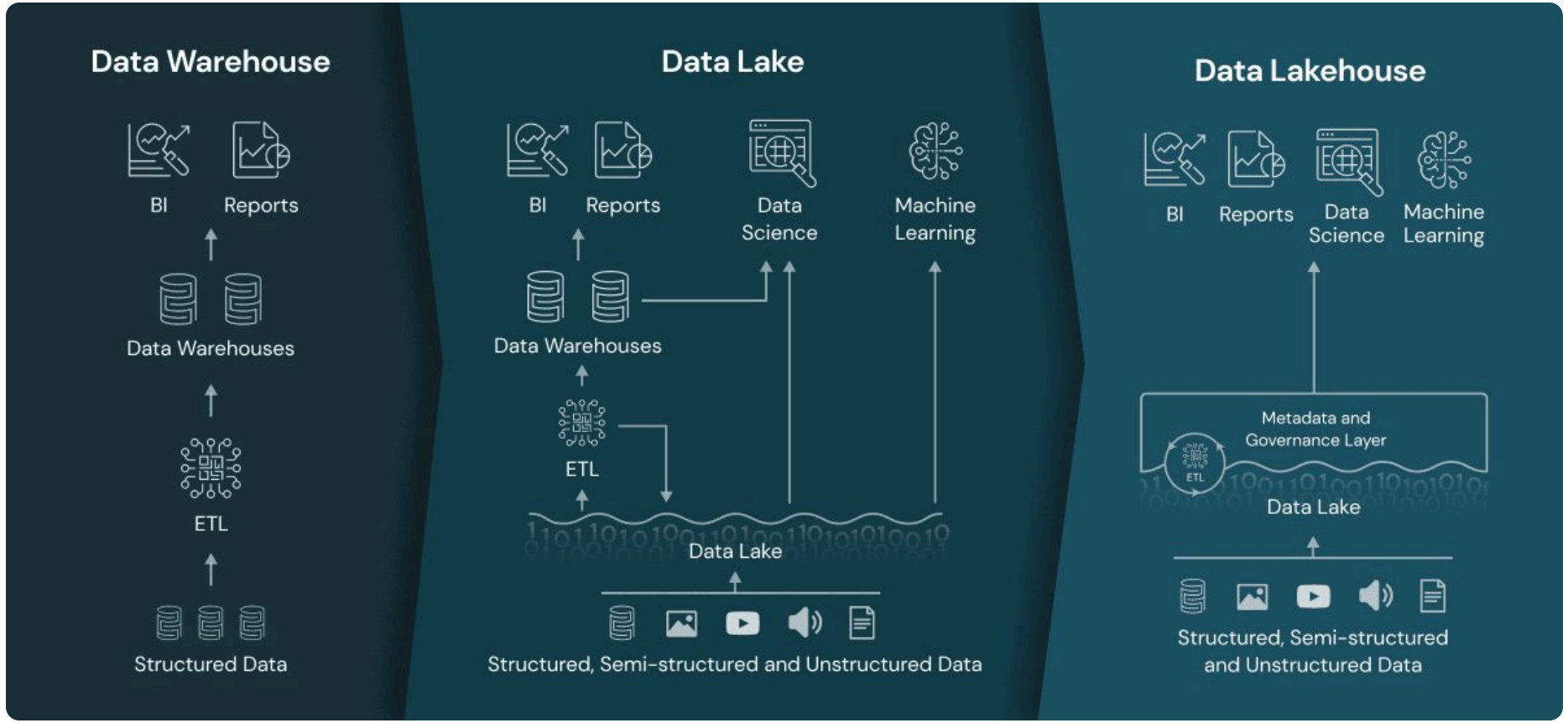


Transform before loading, or load detail first and transform inside the warehouse

§ Lakehouse Management

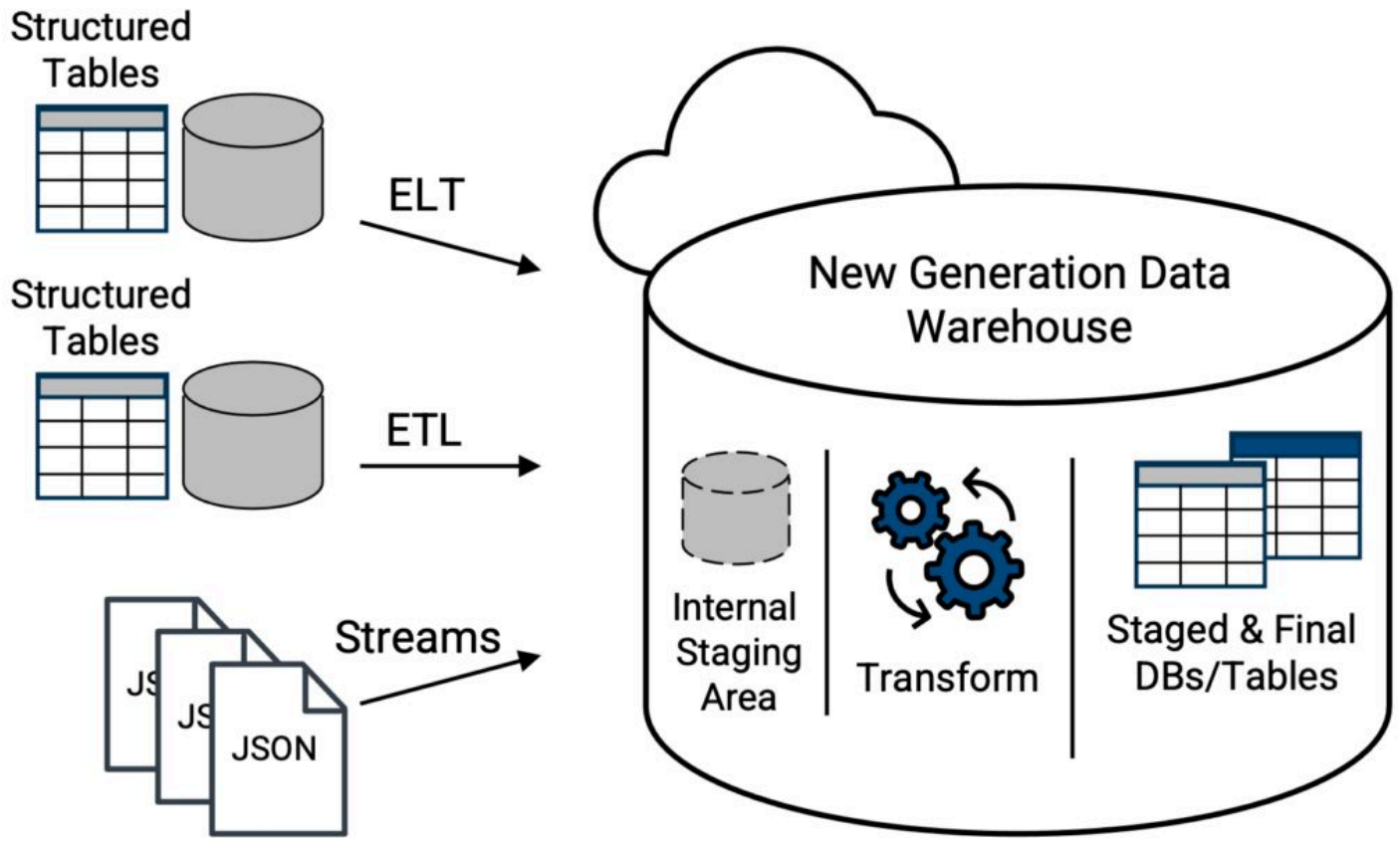


Why a Lakehouse



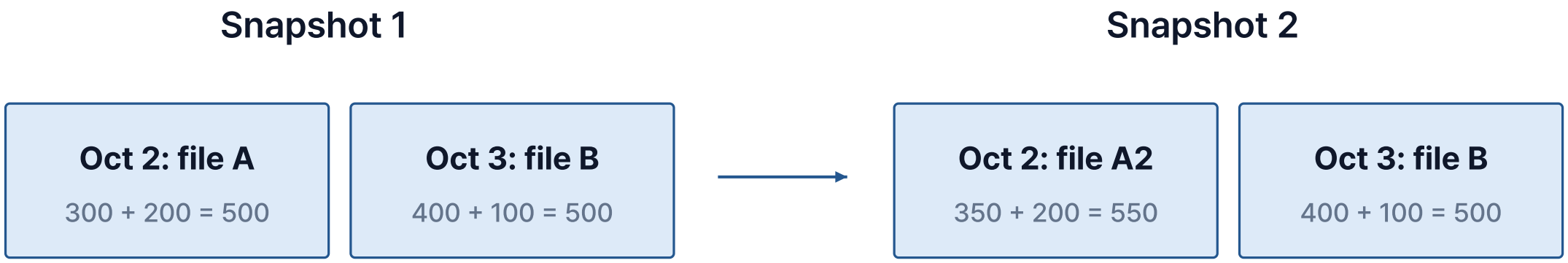
Lakehouse adds table management over lake files for shared analytical use

Lakehouse Data Preparation



A unified architecture still needs staging, transformations and curated analytical tables

Consistent Table Snapshots



Commit a complete file list; retain unchanged file B

A new snapshot changes October 2 while retaining the unchanged October 3 file

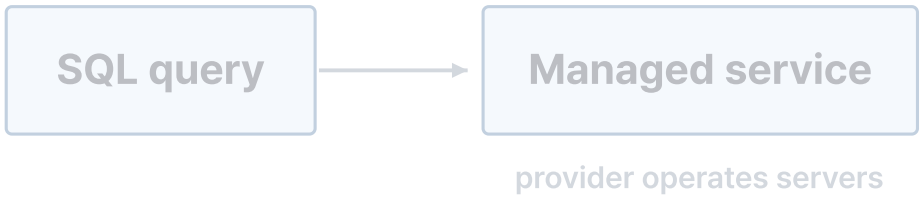
Tools Around a Lakehouse

Layer	Examples	Role
Object storage	S3, Cloud Storage	Store the data files
Columnar file format	Parquet	Encode values in files
Table format	Iceberg, Delta Lake	Manage table metadata and changes
Compute and platform	Spark, Databricks	Execute transformations and analytical queries

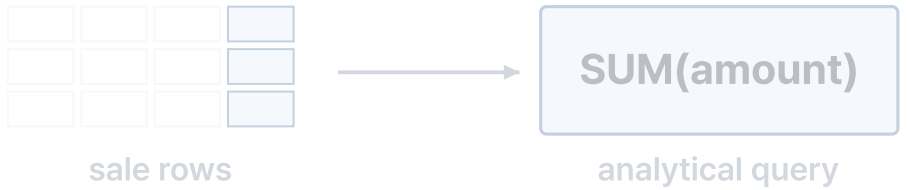
Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

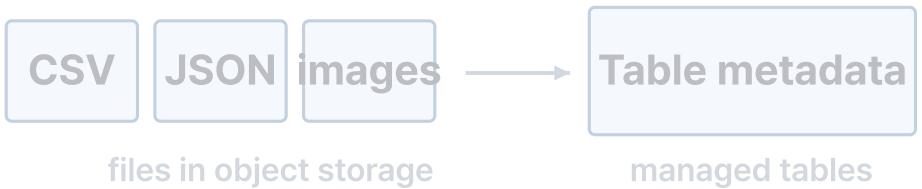
01 Cloud and Serverless



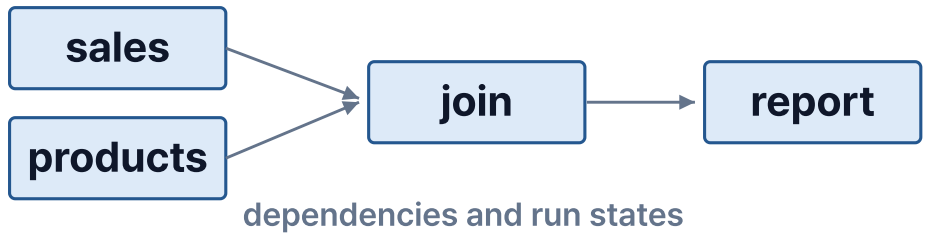
02 Analytical Warehouses



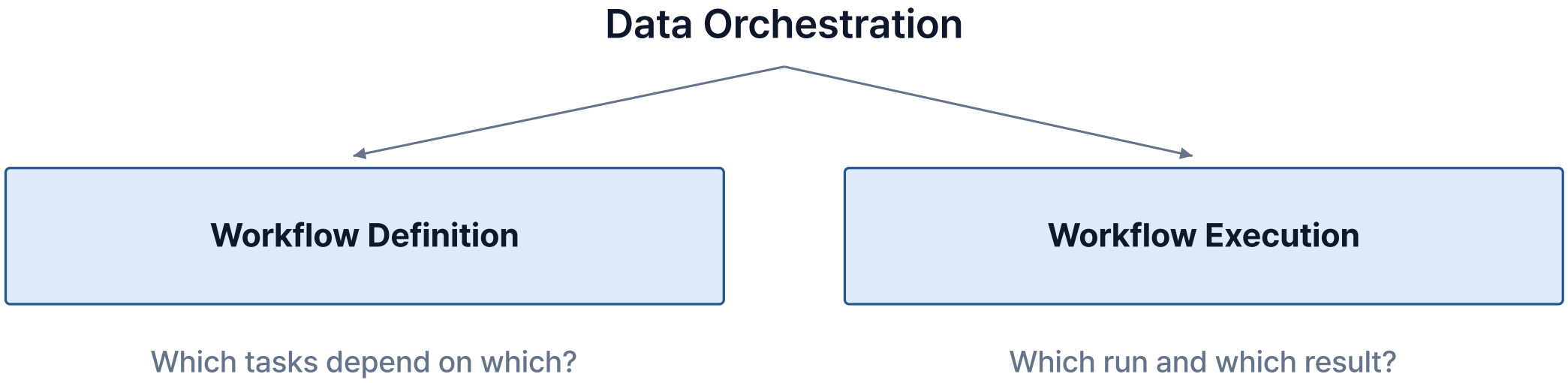
03 Lakes and Lakehouses



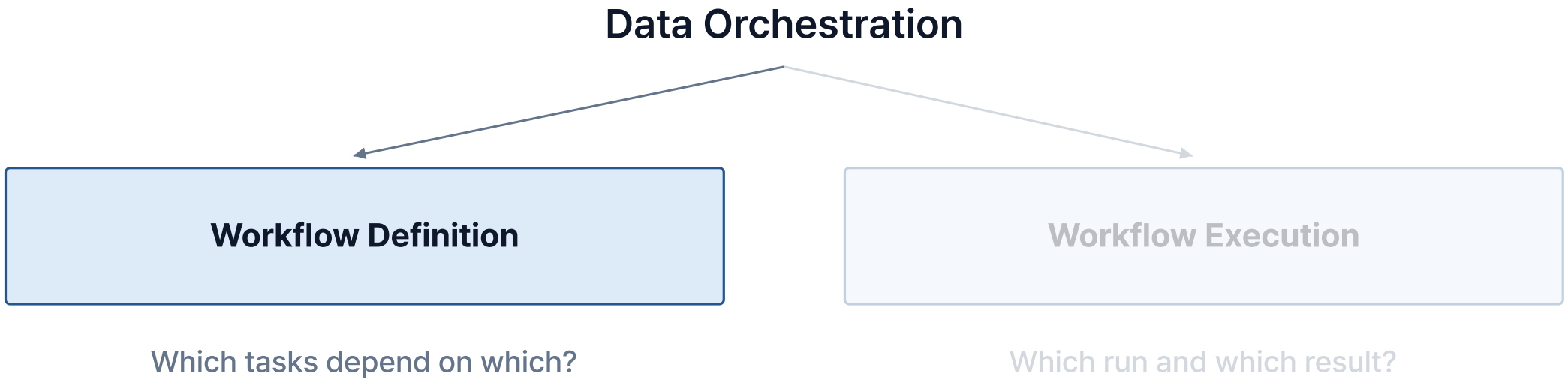
04 Data Orchestration



Data Orchestration

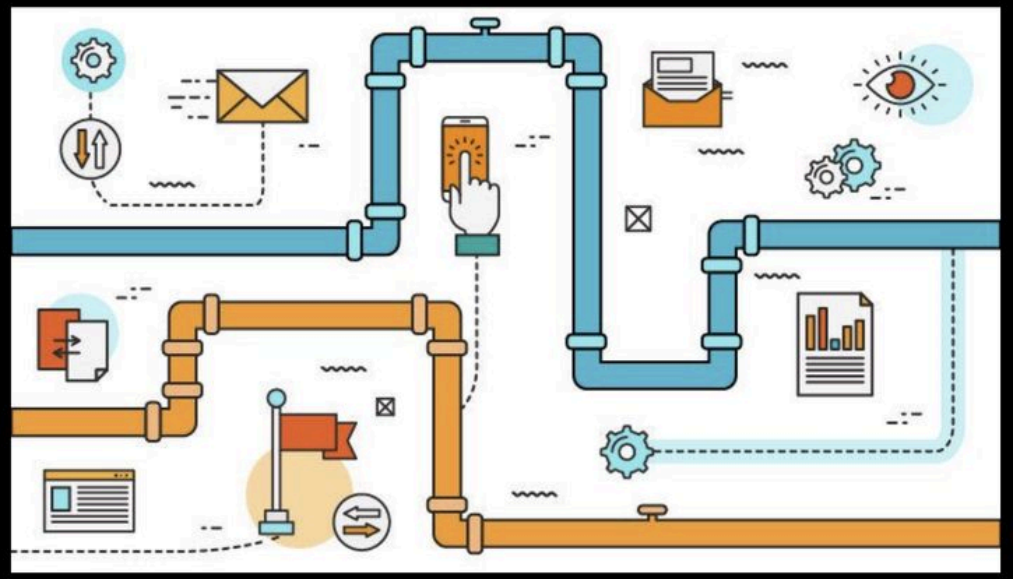


§ Workflow Definition



Why Data Orchestration

Data **orchestration** is the process of taking siloed data from multiple data storage locations, combining and organizing it, and making it available for data analysis tools. It can also include tasks like provisioning resources and monitoring.

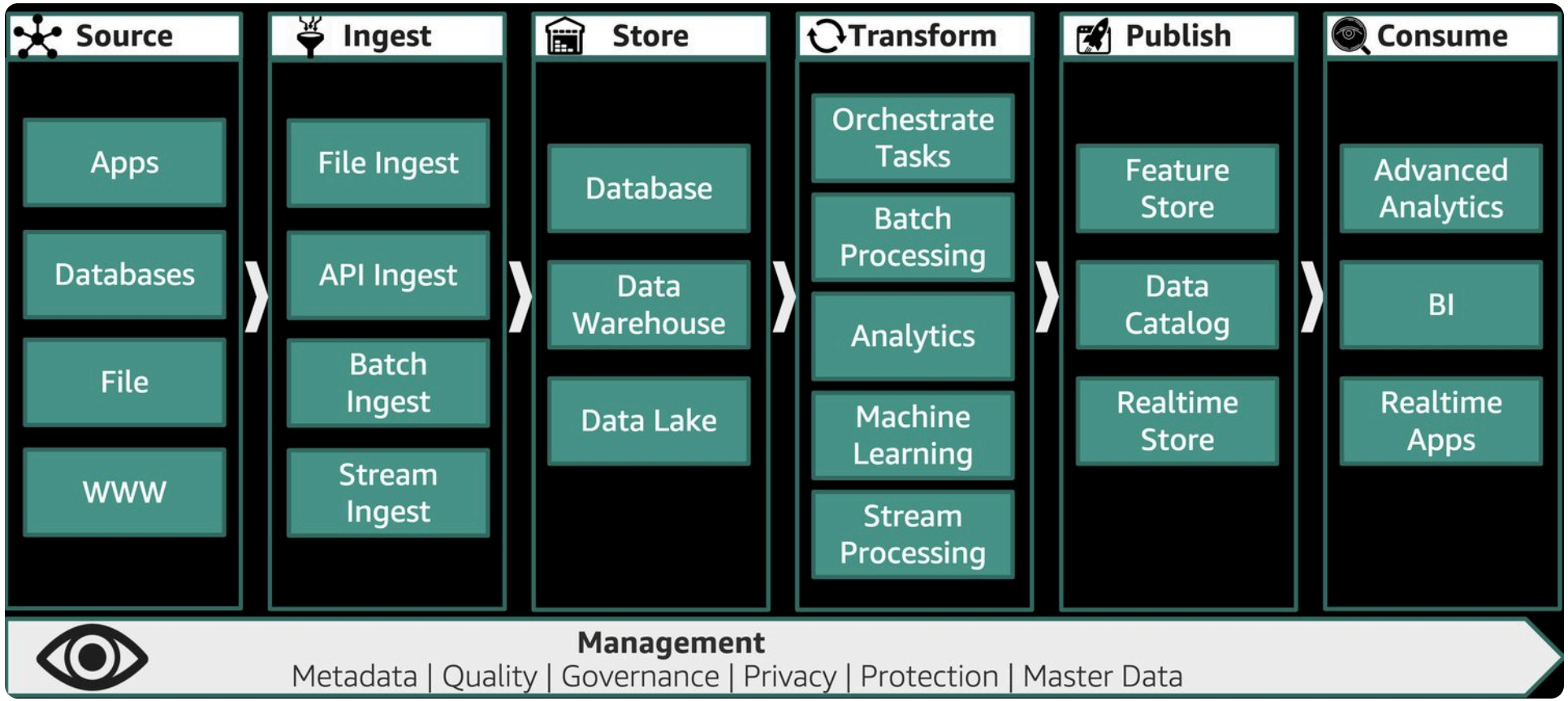


Data Pipelines, Workflows, Jobs



A successful query can still run before its required data is ready

A Data Pipeline



A pipeline carries data; orchestration coordinates the work that moves and transforms it

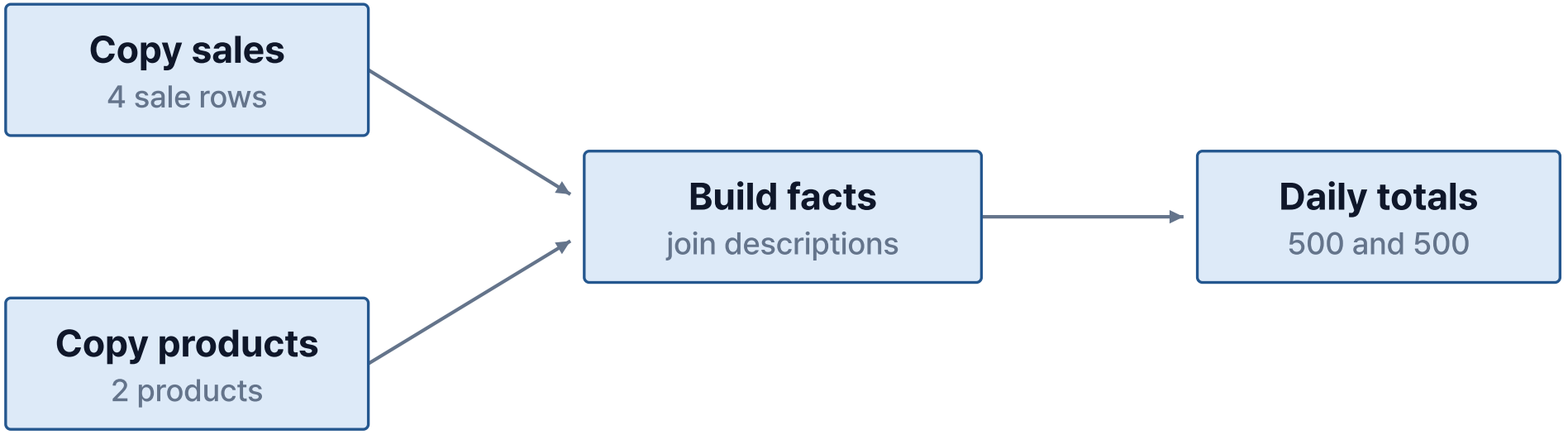
Airflow at Airbnb

Year	Development
2014	Maxime Beauchemin starts Airflow at Airbnb
2015	Airbnb announces the open-source project
2016	The project enters Apache incubation
2019	Apache Airflow becomes a top-level project

Airflow grew from a company workflow tool into an Apache project

A Sales Workflow DAG

A Directed Acyclic Graph (DAG) defines tasks and their dependencies



A task starts after its required inputs succeed

Sales and products can load independently; the join waits for both inputs

Dependencies in Python

Airflow dependency notation, with task definitions omitted

```
copy_sales >> build_facts
copy_products >> build_facts
build_facts >> daily_totals
```

Both copy tasks are upstream of build_facts

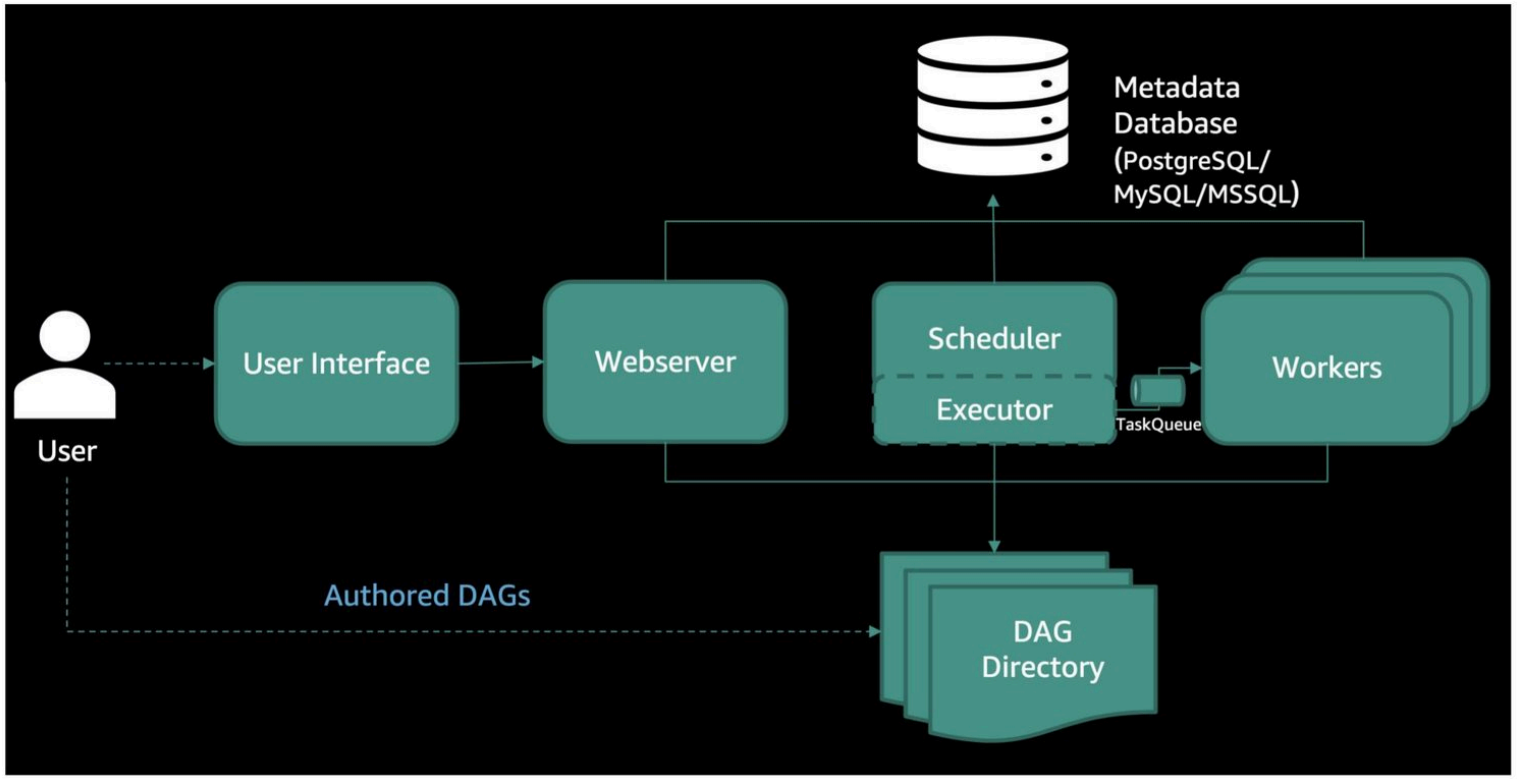
Tasks, Operators and Sensors

Concept	Meaning	Sales workflow example
Task	One defined unit of work	Load the sales file
Operator	A template used to create a task	Submit a SQL query
Sensor	A waiting task pattern	Wait for the input file

A workflow combines concrete tasks built from reusable execution and waiting patterns

Airflow Control and Workers

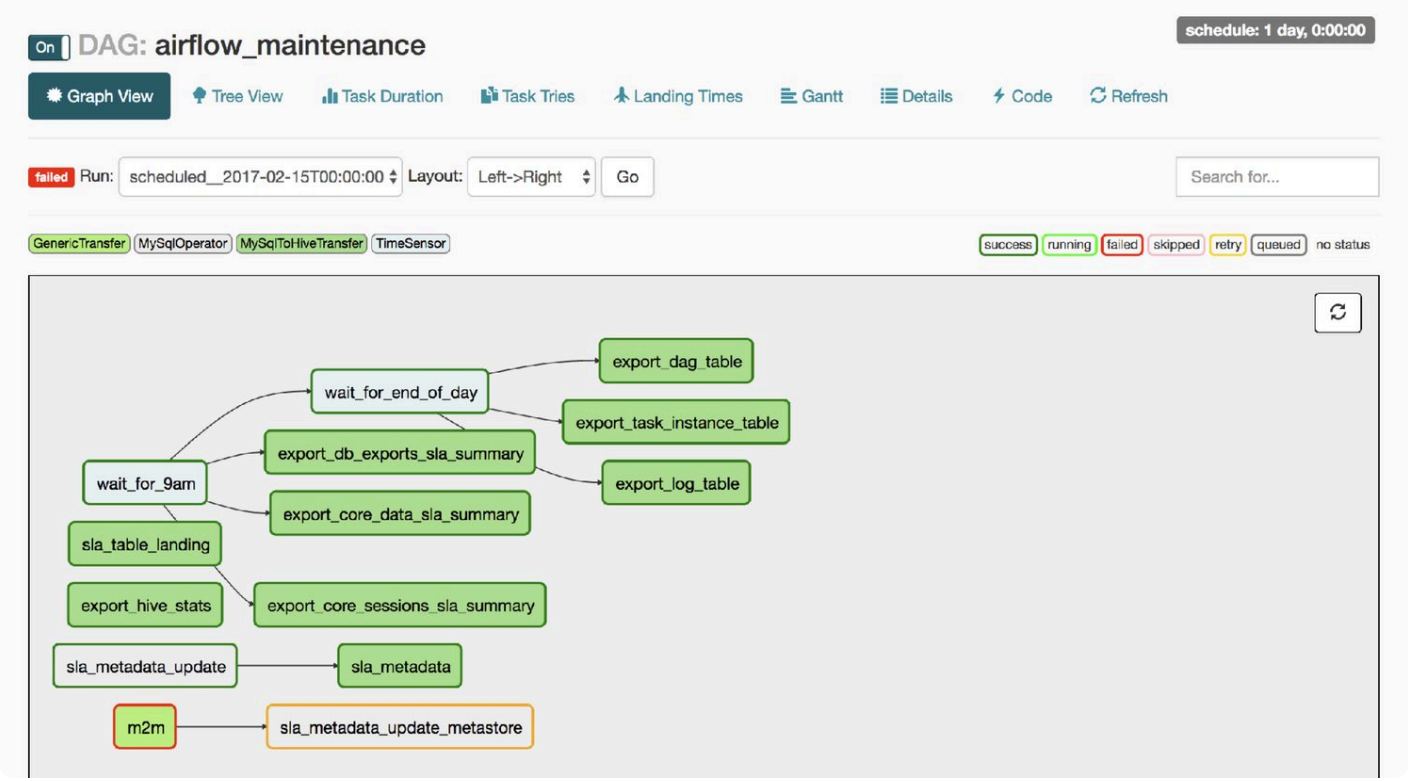
Historical architecture: focus on the responsibilities, rather than deployment details



Scheduling, task execution and recorded state are distinct responsibilities

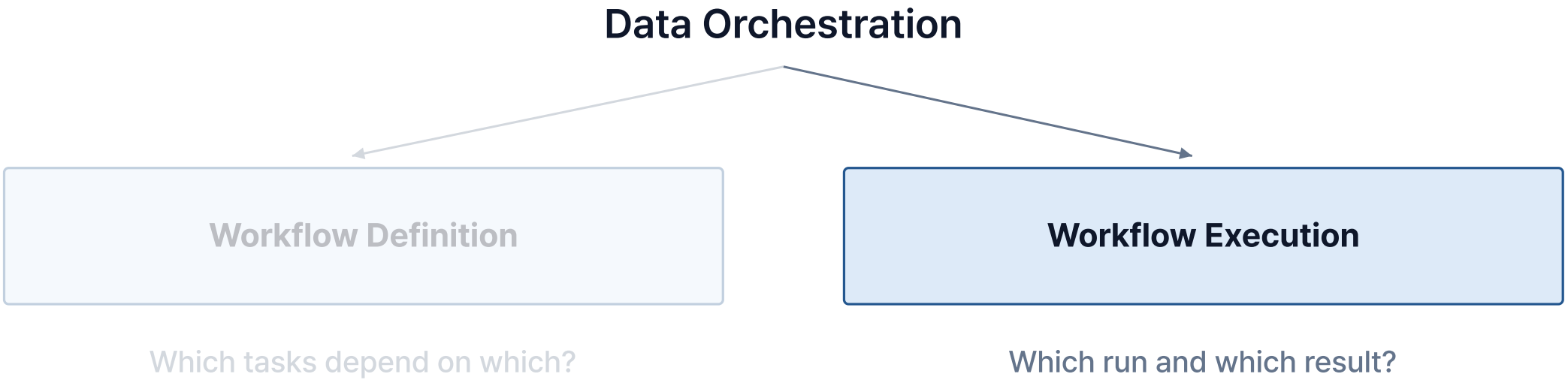
Airflow Graph View

Historical Airflow interface



A real workflow branches into independent tasks and joins dependent results

§ Workflow Execution



Caveat: An Upstream Failure

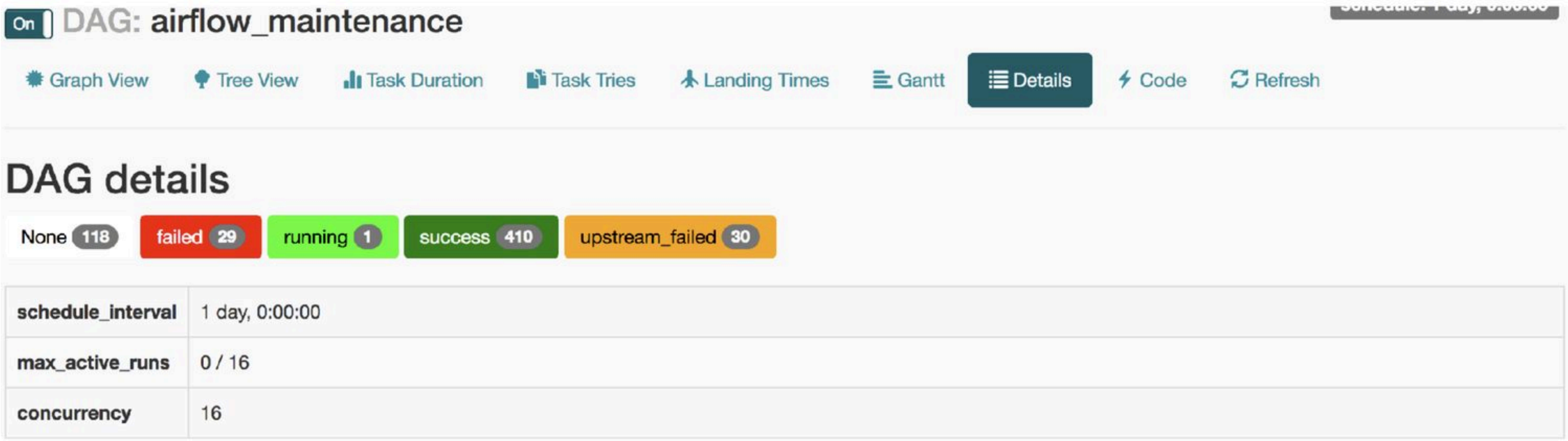


A task starts after its required inputs succeed

A failed product load prevents the dependent join from using incomplete inputs

Task States in Airflow

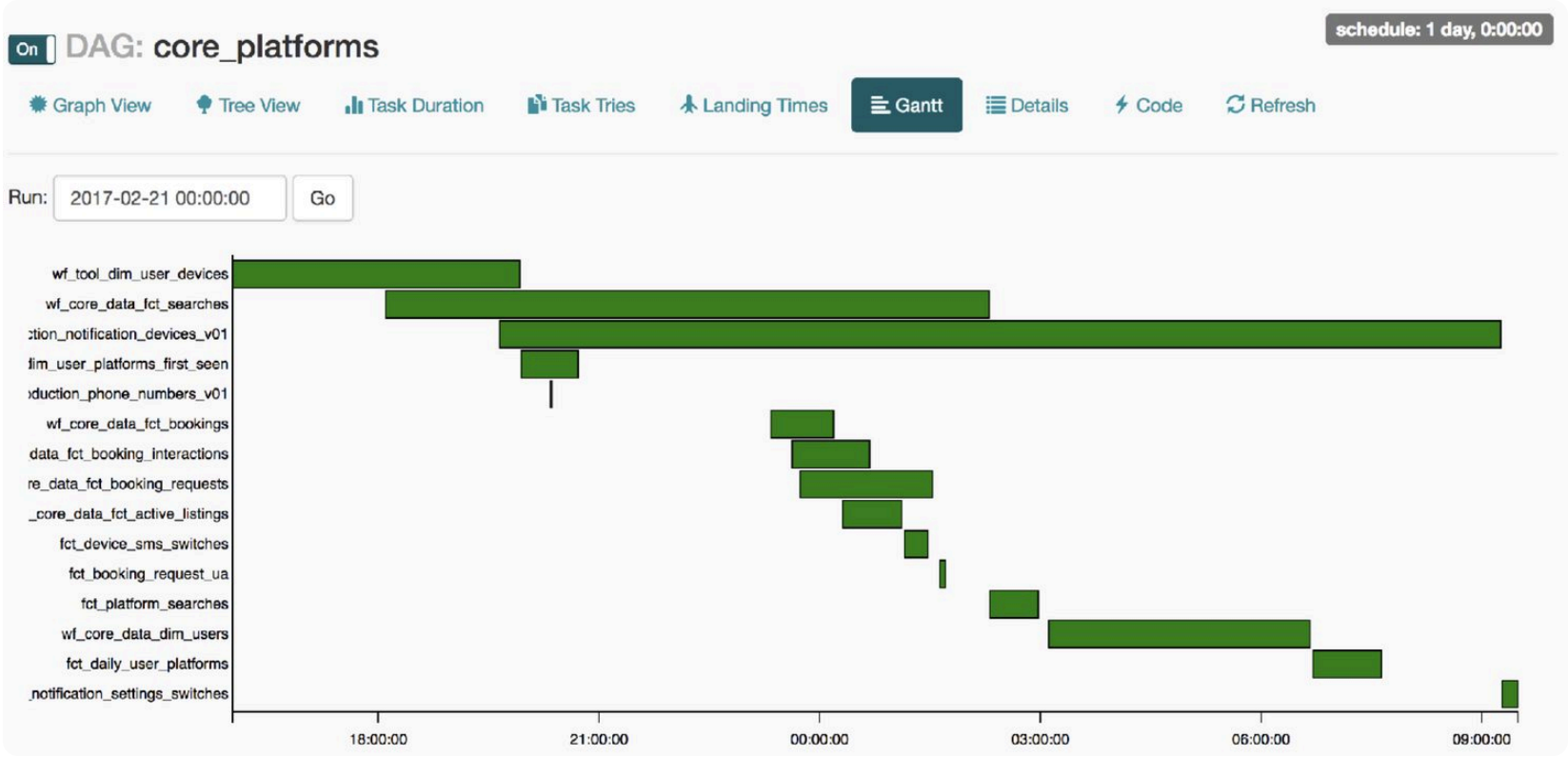
Historical Airflow interface



A workflow run can contain successful, failed and blocked tasks

Task Durations and Overlap

Historical Airflow interface



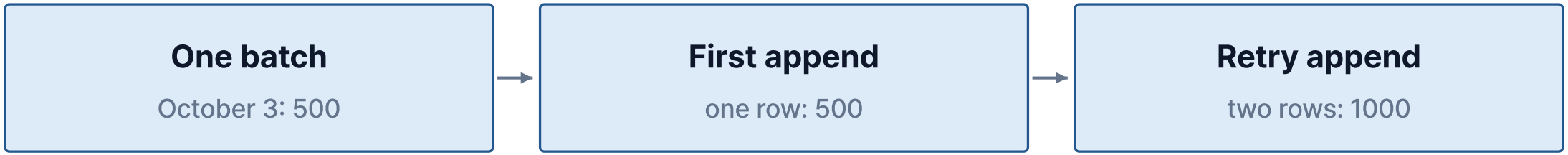
Overlapping task bars reveal concurrency; long dependent paths can delay completion

Data Interval and Run Time

Data being processed	Execution time	Meaning
October 2 sales	October 3 morning	Scheduled daily processing
October 2 sales	October 5 afternoon	Reprocess a corrected past date
October 3 sales	October 5 afternoon	Process another date in the same recovery

The date being recomputed can differ from the time when the task runs

Caveat: Retrying an Append



Retrying the same append can count the same sales twice

Demo: Repeatable Date Load

Local SQLite exercise: safe task behavior under retries

1. Run the supplied date replacement twice in one script
2. Compare it with appending the same date twice

```
BEGIN;
DELETE FROM daily_sales WHERE
sale_date='2025-10-03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY
sale_date;
COMMIT;
SELECT * FROM daily_sales ORDER BY sale_date;
```

QUERY RESULT

sale_date	total
2025-10-02	500
2025-10-03	500

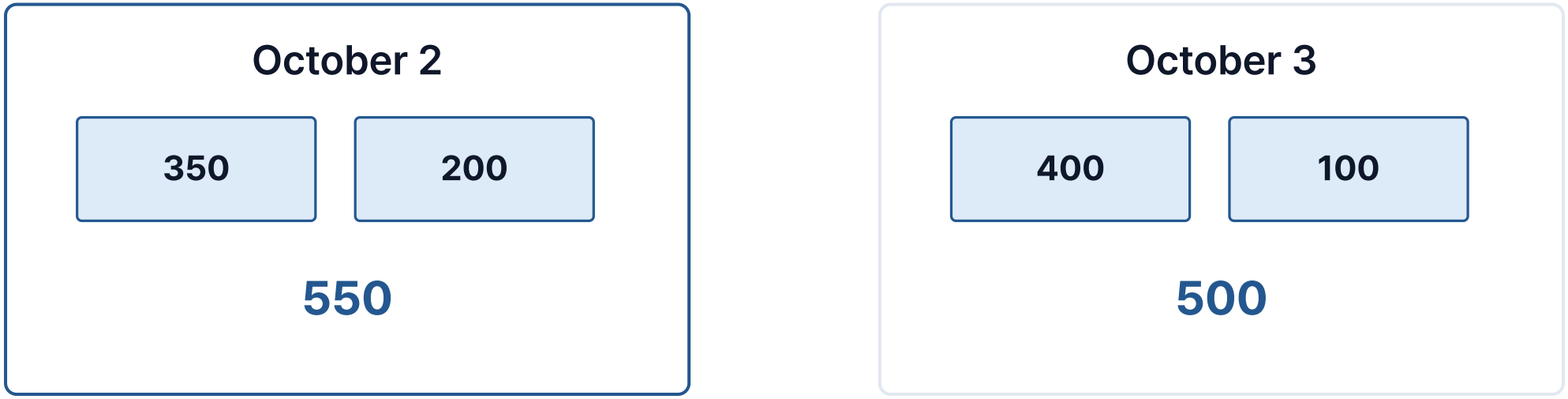
Answer: Repeatable Date Load

```
BEGIN;
DELETE FROM daily_sales WHERE sale_date='2025-10-
03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY sale_date;
COMMIT;
BEGIN;
DELETE FROM daily_sales WHERE sale_date='2025-10-
03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY sale_date;
COMMIT;
SELECT * FROM daily_sales ORDER BY sale_date;
```

sale_date	total
2025-10-02	500
2025-10-03	500

Two replacements leave one correct total for each date

Backfill After a Correction



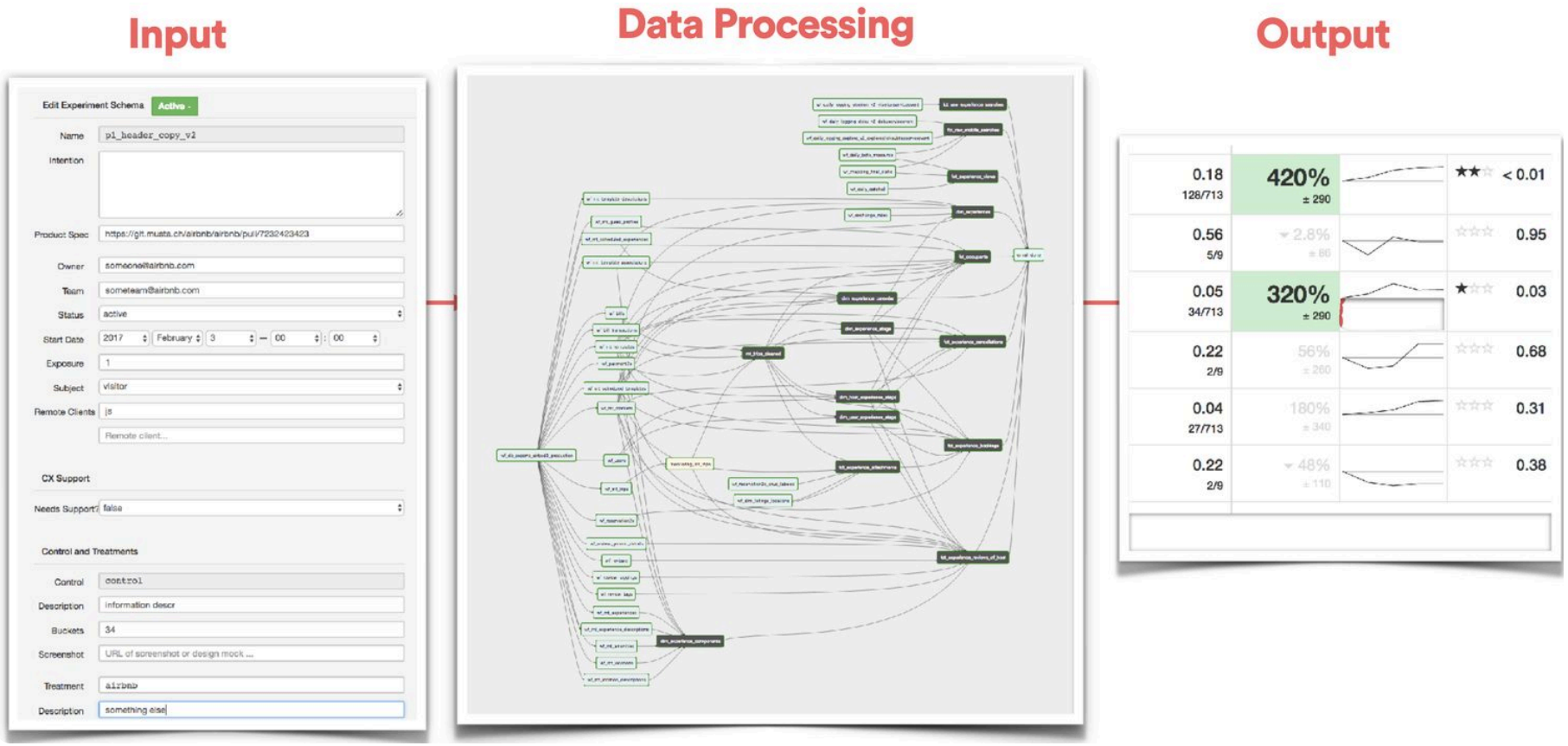
Recomputing October 2 changes its total to 550 while October 3 remains 500

Success and Data Checks

Check	Expected for this report	Failure caught
One row per sale_date	2 distinct dates, 2 rows	Repeated date rows
Reconcile to source	500 for each date	Missing or repeated sales
Publish after checks	Both checks succeed	A successful query with wrong data

Execution success and valid analytical data are separate conditions

The Complete Workflow



Orchestration connects ingestion, distributed computation and published analytical results

Takeaway: Tools Have Different Responsibilities

1. **Cloud and serverless** move infrastructure operations behind managed interfaces
2. **Warehouses and lakehouses** organize data for reliable analytical access
3. **Orchestration** connects tasks, run states and safe reprocessing